

SOLIDWORKS API (.NET VBA C#) — Examples (34/50): **Get_Template_Sheet_Metal_Feature_Data_Ex-** **Get_Visible_Drawing_Components_Example_C**

SOLIDWORKS API Documentation (converted from CHM)

Get Sheet Metal Feature Data Example (VBA)

This example shows how to get the sheet metal feature data from a sheet metal part created in SOLIDWORKS 2013 or later.

```
'-----  
' Preconditions:  
' 1. Open a sheet metal part created in SOLIDWORKS 2013 or later.  
' 2. Open an Immediate window.  
'  
' Postconditions: Inspect the Immediate window to see whether  
' a fixed face reference is selected.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelExt As SldWorks.ModelDocExtension  
    Dim swSelData As SldWorks.SelectData  
    Dim swSheetMetalTemplFeature As SldWorks.Feature  
    Dim swSheetMetal As SldWorks.SheetMetalFeatureData  
    Dim swFixedFace As SldWorks.Face2  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swModelExt = swModel.Extension  
  
    Set swSheetMetalTemplFeature = swModelExt.GetTemplateSheetMetal  
    Set swSheetMetal = swSheetMetalTemplFeature.GetDefinition  
  
    ' Roll back the model  
    bRet = swSheetMetal.AccessSelections(swModel, Nothing)  
    Set swFixedFace = swSheetMetal.FixedReference  
  
    If Not swFixedFace Is Nothing Then  
        Debug.Print "A fixed face or fixed edge is selected."  
        bRet = swFixedFace.Select4(False, swSelData)  
    Else  
        ' A fixed face or fixed edge is not required,  
        ' so Null is a valid value  
        Debug.Print "There is no fixed face or fixed edge in this sheet metal part."  
    End If  
  
    ' Cancel changes and roll forward the model  
    swSheetMetal.ReleaseSelectionAccess  
End Sub
```

Get Sheet Metal Feature Data Example (VB.NET)

This example shows how to get the sheet metal feature data from a sheet metal part created in SOLIDWORKS 2013 or later.

```
'-----  
' Preconditions:  
' 1. Open a sheet metal part created in SOLIDWORKS 2013 or later.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Inspect the Immediate window to see whether a fixed face reference is selected.  
' 2. Inspect the selection in the graphics window.  
' 3. Press F5 to finish the macro.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelExt As ModelDocExtension  
        Dim swSelData As SelectData = Nothing  
        Dim swSheetMetalTemplFeature As Feature  
        Dim swSheetMetal As SheetMetalFeatureData  
        Dim swFixedFace As Face2  
        Dim swEntity As Entity  
        Dim bRet As Boolean  
  
        swModel = swApp.ActiveDoc  
        swModelExt = swModel.Extension  
  
        swSheetMetalTemplFeature = swModelExt.GetTemplateSheetMetal  
        swSheetMetal = swSheetMetalTemplFeature.GetDefinition  
  
        ' Roll back the model  
        bRet = swSheetMetal.AccessSelections(swModel, Nothing)  
        swFixedFace = swSheetMetal.FixedReference  
  
        If Not swFixedFace Is Nothing Then  
  
            Debug.Print("A fixed face or fixed edge is selected.")  
            swEntity = swFixedFace  
            bRet = swEntity.Select4(False, swSelData)  
            Stop  
  
        Else  
  
            ' A fixed face or fixed edge is not required,  
            ' so Null is a valid value  
            Debug.Print("There is no fixed face or fixed edge in this sheet metal part.")  
  
        End If  
  
        ' Cancel changes and roll forward the model  
        swSheetMetal.ReleaseSelectionAccess()  
  
    End Sub  
  
Public swApp As SldWorks
```

End Class

Get Temporary Axes in Each Drawing View Example (VBA)

This example shows how to get the temporary axes in each drawing view and convert them to lines.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\block.sldprt.  
' 2. Click File > Make Drawing from Part and  
'    add one or more views to the drawing.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Converts all temporary axes to lines.  
' 2. Examine the Immediate window and graphics area.  
'-----
```

Option Explicit

Dim swApp As SldWorks.SldWorks

Dim swDrawDoc As SldWorks.DrawingDoc

Dim swSelMgr As SldWorks.SelectionMgr

Dim swView As SldWorks.View

Dim pline As SldWorks.SketchSegment

Dim itr As Long

Sub main()

Set swApp = Application.SldWorks

Set swDrawDoc = swApp.**ActiveDoc**

Set swSelMgr = swDrawDoc.**SelectionManager**

' Iterate through the views

Set swView = swDrawDoc.**GetFirstView**

While Not swView Is Nothing

If Not swView.**Type** = swDrawingSheet Then

Debug.Print swView.**GetName2**

swDrawDoc.**ActivateView** swView.**GetName2**

Dim nCount As Long, vTempAxesPoints As Variant, nJump As Long

' Get number of temporary axes

nCount = swView.**GetTemporaryAxesCount**

Debug.Print " Number of temporary axes = " & nCount

nJump = 6

' Get temporary axes

vTempAxesPoints = swView.**GetTemporaryAxes**

For itr = 0 To nCount - 1

Dim startPt(2) As Double

Dim endPt(2) As Double

startPt(0) = vTempAxesPoints(nJump * itr)

startPt(1) = vTempAxesPoints(nJump * itr + 1)

startPt(2) = vTempAxesPoints(nJump * itr + 2)

```
endPt(0) = vTempAxesPoints(nJump * itr + 3)
endPt(1) = vTempAxesPoints(nJump * itr + 4)
endPt(2) = vTempAxesPoints(nJump * itr + 5)
```

```
Debug.Print "  Temporary axis " & itr; " data :"  
Debug.Print "    start : x = " & startPt(0)  
Debug.Print "        y = " & startPt(1)  
Debug.Print "        z = " & startPt(2)  
Debug.Print "    end   : x = " & endPt(0)  
Debug.Print "        y = " & endPt(1)  
Debug.Print "        z = " & endPt(2)
```

```
    ' Convert temporary axes to line
```

```
Set pline = swDrawDoc.CreateLine2(startPt(0), startPt(1), startPt(2), endPt(0), endPt(1), endPt(2))
```

```
If Not pline Is Nothing Then
```

```
    ' Color
```

```
    Dim clr As Long
```

```
    clr = swApp.GetUserPreferenceIntegerValue(swSystemColorsConstructionGeometry)
```

```
    pline.Color = clr
```

```
    ' Width
```

```
    pline.Width = swLW_NORMAL
```

```
    ' Style ( only an approximation )
```

```
    pline.Style = swLineCHAIN
```

```
End If
```

```
Next itr
```

```
End If
```

```
    ' Get next view
```

```
Set swView = swView.GetNextView
```

```
Wend
```

```
swDrawDoc.ClearSelection2 True
```

```
End Sub
```

Get Temporary Axis and Its Reference Face Example (C#)

This example shows how to get a temporary axis and its reference face.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\box.sldprt.
// 2. Click View > Hide/Show > Temporary Axes.
// 3. Select the temporary axis.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Gets whether the selected entity is a temporary
//    axis.
// 2. Gets the reference face of the temporary axis.
// 3. Examine the Immediate window and graphics area.
//
// NOTE: Because the part is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            Entity swEntity = default(Entity);
            RefAxis swRefAxis = default(RefAxis);
            object obj = null;
            Face2 swFace = default(Face2);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Get selected entity
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swEntity = (Entity)swFeature;

            if (swEntity.GetType() == (int)swSelectType_e.swSelDATUMAXES)
            {
                swRefAxis = (RefAxis)swFeature.GetSpecificFeature2();
                //Get whether selected entity is a temporary axis
                if (swRefAxis.IsTempAxis())
                {
                    Debug.Print("Is temporary reference axis");
                    //Get reference face of temporary axis
                    obj = (object)swRefAxis.GetTempAxisReferenceFace();
                    if ((obj != null))
                    {
                        Debug.Print("Got reference face of temporary axis");
                        swFace = (Face2)obj;
                        swFace.Highlight(true);
                        Debug.Print("Highlighted reference face of temporary axis; examine the graphics area");
                    }
                    else
                    {
                        Debug.Print("Did not get reference face of temporary axis");
                    }
                }
                else
                {
                    Debug.Print("Not a temporary axis");
                }
            }
            else
            {
                Debug.Print("Select a temporary axis and run the macro again");
            }
        }
    }
}

/// <summary>
```

```
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}
```

Get Temporary Axis and Its Reference Face Example (VBA)

This example shows how to get a temporary axis and its reference face.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\box.sldprt.  
' 2. Click View > Hide/Show > Temporary Axes.  
' 3. Select the temporary axis.  
' 4. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets whether the selected entity is a temporary  
'    axis.  
' 2. Gets the reference face of the temporary axis.  
' 3. Examine the Immediate window and graphics area.  
'  
' NOTE: Because the part is used elsewhere, do not save  
' changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeature As SldWorks.Feature  
    Dim swEntity As SldWorks.Entity  
    Dim swRefAxis As SldWorks.RefAxis  
    Dim obj As Object  
    Dim swFace As SldWorks.Face2  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    'Get selected entity  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)  
    Set swEntity = swFeature  
  
    If swEntity.GetType = swSelectType_e.swSelDATUMAXES Then  
        Set swRefAxis = swFeature.GetSpecificFeature2  
        'Get whether selected entity is a temporary axis  
        If swRefAxis.IsTempAxis Then  
            Debug.Print "Is temporary reference axis"  
            'Get reference face of temporary axis  
            Set obj = swRefAxis.GetTempAxisReferenceFace  
            If Not obj Is Nothing Then  
                Debug.Print "  Got reference face of temporary axis"  
                Set swFace = obj  
                swFace.Highlight (True)  
                Debug.Print "  Highlighted reference face of temporary axis; examine the graphics area"  
            Else  
                Debug.Print "Did not get reference face of temporary axis"  
            End If  
        Else  
            Debug.Print "Not a temporary axis"  
        End If  
    Else  
        Debug.Print "Select a temporary axis and run the macro again"  
    End If  
  
End Sub
```

Get_Temporary_Axis_and_Its_Reference_Face_Example_VBNET (Topic)

SOLIDWORKS API Help

Get Temporary Axis and Its Reference Face Example (VB.NET)

This example shows how to get a temporary axis and its reference face.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\box.sldprt.
' 2. Click View > Hide/Show > Temporary Axes.
' 3. Select the temporary axis.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Gets whether the selected entity is a temporary
'    axis.
' 2. Gets the reference face of the temporary axis.
' 3. Examine the Immediate window and graphics area.
'
' NOTE: Because the part is used elsewhere, do not save
'       changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swEntity As Entity
        Dim swRefAxis As RefAxis
        Dim obj As Object
        Dim swFace As Face2

        swModel = swApp.ActiveDoc

        'Get selected entity
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swEntity = swFeature

        If swEntity.GetType = swSelectType_e.swSelDATUMAXES Then
            swRefAxis = swFeature.GetSpecificFeature2
            'Get whether selected entity is a temporary axis
            If swRefAxis.IsTempAxis Then
                Debug.Print("Is temporary reference axis")
                'Get reference face of temporary axis
                obj = swRefAxis.GetTempAxisReferenceFace
                If Not obj Is Nothing Then
                    Debug.Print(" Got reference face of temporary axis")
                    swFace = obj
                    swFace.Highlight(True)
                    Debug.Print(" Highlighted reference face of temporary axis; examine the graphics area")
                Else
                    Debug.Print("Did not get reference face of temporary axis")
                End If
            Else
                Debug.Print("Not a temporary axis")
            End If
        Else
            Debug.Print("Select a temporary axis and run the macro again")
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
End Class
```

Get Text Items in GTol Frame Example (C#)

This example shows how to get text items, values, and symbols in a GTol frame.

```
//-----
// Preconditions:
// 1. Open document with a GTol frame and select that GTol
//    frame.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the text items, values, and symbols in the
//    selected GTol frame.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            SelectionMgr swSelMgr;
            GTol swGtol;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swGtol = (GTol)swSelMgr.GetSelectedObject6(1, -1);

            Debug.Print("GetTextCount = " + swGtol.GetTextCount().ToString());

            for (int idx = 0; idx < swGtol.GetTextCount(); idx++)
            {
                Debug.Print("GetTextAtIndex(" + idx.ToString() + ") = " + swGtol.GetTextAtIndex(idx));
            }
            int count = 0;
            count = swGtol.GetFrameCount();
            count = count - 1;
            Debug.Print("GetFrameCount = " + count.ToString());
            for (int idx = 1; idx <= (swGtol.GetFrameCount() - 1); idx++)
            {
                Object myParams;
                Object mySymbols;
                myParams = swGtol.GetFrameValues((short)idx);
                if (myParams != null)
                {
                    Array myParamArray = (Array)myParams;
                    Debug.Print("GetFrameValues(" + idx.ToString() + ") = " + myParamArray.GetValue(0) + "," + myParam
Array.GetValue(1) + "," + myParamArray.GetValue(2) + "," + myParamArray.GetValue(3) + "," + myParamArray.GetValue(4));
                    mySymbols = swGtol.GetFrameSymbols3((short)idx);
                    Array mySymbolsArray = (Array)mySymbols;
                    Debug.Print(" GetFrameSymbols3(" + idx.ToString() + ") = " + mySymbolsArray.GetValue(0) + "," + m
ySymbolsArray.GetValue(1) + "," + mySymbolsArray.GetValue(2) + "," + mySymbolsArray.GetValue(3) + "," + mySymbolsArray
.GetValue(4) + "," + mySymbolsArray.GetValue(5));
                }
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Text Items in GTol Frame Example (VB.NET)

This example shows how to get text items, values, and symbols in a GTol frame.

```
'-----
' Preconditions:
' 1. Open a document with a GTol frame and select that GTol
'    frame.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Gets the text items, values, and symbols in the
'    selected GTol frame.
' 2. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics
Imports System

Partial Public Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swGtol As GTol

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        swGtol = swSelMgr.GetSelectedObject6(1, -1)

        Debug.Print("GetTextCount = " & swGtol.GetTextCount().ToString())

        For idx As Integer = 0 To swGtol.GetTextCount() - 1
            Debug.Print(("GetTextAtIndex(" & idx.ToString() & ") = ") + swGtol.GetTextAtIndex(idx))
        Next

        Debug.Print("GetFrameCount = " & (swGtol.GetFrameCount().ToString()) - 1)

        For idx As Integer = 1 To swGtol.GetFrameCount() - 1
            Dim myParams As Object
            Dim arrSymbols As Object
            myParams = swGtol.GetFrameValues(idx)
            If myParams IsNot Nothing Then
                Debug.Print((((("GetFrameValues(" & idx.ToString() & ") = ") & myParams.GetValue(0) & "," & myParams
                .GetValue(1) & "," & myParams.GetValue(2) & "," & myParams.GetValue(3) & "," & myParams.GetValue(4))
                arrSymbols = swGtol.GetFrameSymbols3(idx)
                Debug.Print(((((" GetFrameSymbols3(" & idx.ToString() & ") = ") & arrSymbols(0) & "," & arrSymbol
                s(1) & "," & arrSymbols(2) & "," & arrSymbols(3) & "," & arrSymbols(4) & "," & arrSymbols(5) & ")")))))
            End If
        Next

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Text Items in GTol Frame Example (VBA)

This example shows how to get text items, values, and symbols in a GTol frame.

```
'-----  
' Preconditions:  
' 1. Open a document with a GTol frame and select that GTol  
'    frame.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Gets the text items, values, and symbols in the  
'    selected GTol frame.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim selGtol As SldWorks.Gtol  
Dim idx As Long  
Dim params As Variant  
Dim arrSymbols As Variant  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
    Set selGtol = swSelMgr.GetSelectedObject6(1, 0)  
  
    Debug.Print "GetTextCount = " & CStr(selGtol.GetTextCount)  
  
    For idx = 0 To selGtol.GetTextCount - 1  
        Debug.Print "GetTextAtIndex(" & CStr(idx) & ") = " & selGtol.GetTextAtIndex(idx)  
    Next idx  
  
    Debug.Print "GetFrameCount = " & (CStr(selGtol.GetFrameCount) - 1)  
  
    For idx = 1 To selGtol.GetFrameCount - 1  
        params = selGtol.GetFrameValues(idx)  
        Debug.Print "GetFrameValues(" & CStr(idx) & ") = " & params(0) & "," & params(1) & "," & params(2) & "," &  
params(3) & "," & params(4)  
        arrSymbols = selGtol.GetFrameSymbols3(idx)  
        Debug.Print "  GetFrameSymbols3(" & CStr(idx) & ") = " & arrSymbols(0) & ", " & arrSymbols(1) & ", " &  
arrSymbols(2) & ", " & arrSymbols(3) & ", " & arrSymbols(4) & ", " & arrSymbols(5) & ")"  
    Next idx  
  
End Sub
```

Get the Normal and Origin of a Reference Plane Using Its Transform Example (VBA)

This example shows how to get the normal and origin of the reference plane using its transform.

NOTE: The trick is to know that the reference plane's transform is the transform that takes a reference plane from its canonical position/orientation to its actual position/orientation. In this example, the canonical position is aligned with the system defined **Front Plane**.

```
' -----
' Preconditions: Open a part document that has these planes:
' * Front Plane
' * Top Plane
' * Right Plane
' * Plane2
' * Plane3
' * Plane4
' * Plane5
'
' Postconditions:
' 1. Creates a 3D sketch (i.e., a line) from each reference plane
'    origin in the direction of the reference plane normal.
' 2. Examine the graphics area and FeatureManager design tree.
' -----
Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swFeature As SldWorks.Feature
    Dim swRefPlane As SldWorks.RefPlane
    Dim swOriginPoint As SldWorks.MathPoint
    Dim swRefPlaneTransform As SldWorks.MathTransform
    Dim swMathUtility As SldWorks.MathUtility
    Dim aPointData(2) As Double
    Dim vPointData As Variant
    Dim swOriginPointOnRefPlane As SldWorks.MathPoint
    Dim swNormalVector As SldWorks.MathVector
    Dim swNormalVectorRefPlane As SldWorks.MathVector
    Dim aVectorData(2) As Double
    Dim vVectorData As Variant
    Dim dScaleFactor As Double
    Dim strRefPlaneName As String
    Dim aRefPlanes(6) As String
    Dim lIdx As Long

    ' Connect to SOLIDWORKS Design
    Set swApp = Application.SldWorks

    ' Get the math utility
    Set swMathUtility = swApp.GetMathUtility

    ' Get active document
    Set swModel = swApp.ActiveDoc

    ' Cast down
    Set swPart = swModel

    ' Fill an array with names of reference planes to visit:
    ' * You can replace this by a loop over all reference planes in the model
    ' * In particular, check the "Front Plane" as this defines the canonical
    '   position and orientation of a reference plane
    aRefPlanes(0) = "Front Plane"
    aRefPlanes(1) = "Top Plane"
    aRefPlanes(2) = "Right Plane"
    aRefPlanes(3) = "Plane2"
    aRefPlanes(4) = "Plane3"
    aRefPlanes(5) = "Plane4"
    aRefPlanes(6) = "Plane5"

    ' Loop over all reference planes of interest
    For lIdx = LBound(aRefPlanes) To UBound(aRefPlanes)
```

```

' Get reference plane feature by name as feature
Set swFeature = swPart.FeatureByName(aRefPlanes(lIdx))
' Get the name
strRefPlaneName = swFeature.Name
' Cast down
Set swRefPlane = swFeature.GetSpecificFeature2
' Get the reference plane transform:
' * This transform takes a reference plane from
' its canonical position/orientation to its
' actual Position / Orientation
' * The canonical position/orientation is aligned
' with the system defined "Front Plane"
Set swRefPlaneTransform = swRefPlane.Transform

' Create a math point that represents the reference plane's origin
' in the canonical position in world coordinates; this is
' (0.0, 0.0, 0.0)

' Create array data
aPointData(0) = 0#
aPointData(1) = 0#
aPointData(2) = 0#

' Turn into a Variant
vPointData = aPointData

' Create a math point
Set swOriginPoint = swMathUtility.CreatePoint(vPointData)

' Transform the reference plane origin from its canonical
' position to its actual position
Set swOriginPointOnRefPlane = swOriginPoint.MultiplyTransform(swRefPlaneTransform)

' Get point data
vPointData = swOriginPointOnRefPlane.ArrayData

' Create a math vector that represents the reference plane's normal in
' the canonical orientation in world coordinates; this is [0.0, 0.0, 1.0]

' Create array data
aVectorData(0) = 0#
aVectorData(1) = 0#
aVectorData(2) = 1#

' Turn into a Variant
vVectorData = aVectorData

' Create a math vector
Set swNormalVector = swMathUtility.CreateVector(vVectorData)

' Now transform the reference plane normal from its canonical
' orientation to its actual orientation
Set swNormalVectorRefPlane = swNormalVector.MultiplyTransform(swRefPlaneTransform)

' Get vector data
vVectorData = swNormalVectorRefPlane.ArrayData

' Visualize

' As the reference plane normal is normalized, it may be bit out of
' proportion; this factor is arbitrary
dScaleFactor = 0.1

' Insert a 3D sketch
swModel.Insert3DSketch2 True

' Create a line from the reference plane origin in the direction
' of the reference plan normal
swModel.CreateLine2 vPointData(0), vPointData(1), vPointData(2), vPointData(0) + dScaleFactor *
vVectorData(0), vPointData(1) + dScaleFactor * vVectorData(1), vPointData(2) + dScaleFactor * vVectorData(2)

' Close the sketch
swModel.Insert3DSketch2 True

' The sketch is still selected
Set swFeature = swModel.SelectionManager.GetSelectedObject6(1, 0)

' Rename it
swFeature.Name = strRefPlaneName & "-Normal"

```

```
        ' Clear selection for the next pass
        swModel.ClearSelection2 True
    Next lIdx
End Sub
```

Get Title Block Tables Example (C#)

This example shows how to get title block tables.

```
//-----
// Preconditions:
// 1. Open a part that does not contain any annotations.
// 2. Verify that the specified table template exists.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Inspect the output in the Immediate window.
// 2. Observe the new title block table feature under Tables
//    in the FeatureManager design tree (if necessary, right-click the
//    part in the FeatureManager design tree, click Hidden Tree
//    Items > Hide/Show Tree Items > Show in the Tables drop-down list).
// 3. Observe the corresponding title block table annotation
//    in the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        TitleBlockTableAnnotation tbtAnno;
        TitleBlockTableAnnotation anno;
        TitleBlockTableAnnotation tabannoObject;
        TitleBlockTableFeature feat;
        TitleBlockTableFeature tabfeatObject;
        object[] annos;
        int I;
        SelectionMgr selMgr;
        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            Debug.Print("Inserting a title block table into the model using a general table template (*.sldtbt)");
            Debug.Print("");
            tbtAnno = Part.Extension.InsertTitleBlockTable("C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\engl
ish\\connector-table.sldtbt", 500, 500);
            feat = tbtAnno.TitleBlockTableFeature;
            Debug.Print("Title block table feature: " + feat.GetFeature().Name);
            string count = null;
            count = feat.GetTableAnnotationCount().ToString();
            Debug.Print("Title block table annotation count: " + count);
            Debug.Print("Title block table annotations");
            annos = (Object[])feat.GetTableAnnotations();
            for (I = 0; I <= annos.GetUpperBound(0); I++)
            {
                anno = (TitleBlockTableAnnotation)annos[I];
                Debug.Print(" Title block table feature: " + anno.TitleBlockTableFeature.GetFeature().Name);
            }
            Debug.Print("");

            Debug.Print("Selecting title block table feature through SelectByID2 type, TITLEBLOCKTABLEFEAT");
            boolstatus = Part.Extension.SelectByID2(feat.GetFeature().Name, "TITLEBLOCKTABLEFEAT", 0, 0, 0, false, 0,
null, 0);

            Debug.Print(" Casting selected object to ITitleBlockTableFeature");
            selMgr = (SelectionMgr)Part.SelectionManager;
            tabfeatObject = (TitleBlockTableFeature)selMgr.GetSelectedObject6(1, -1);
            Debug.Print(" Title block table feature: " + tabfeatObject.GetFeature().Name);
            Debug.Print("");

            Debug.Print("Selecting title block table annotation through SelectByID2 type, ANNOTATIONTABLES");
            boolstatus = Part.Extension.SelectByID2("DetailItem1@Annotations", "ANNOTATIONTABLES", -0.1205280774849, -
0.01199819470702, 0.04087038255709, false, 0, null, 0);
            Debug.Print(" Casting selected object to ITitleBlockTableAnnotation type...");
            tabannoObject = (TitleBlockTableAnnotation)selMgr.GetSelectedObject6(1, -1);
            Debug.Print(" Getting title block table feature from the title block table annotation");
            Debug.Print(" Title block table feature: " + tabannoObject.TitleBlockTableFeature.GetFeature().Name);
            ;

            Debug.Print("");

            Debug.Print(" Casting selected object to ITableAnnotation type");
            TableAnnotation annoObject = default(TableAnnotation);
            annoObject = (TableAnnotation)selMgr.GetSelectedObject6(1, -1);
        }
    }
}
```

```

swTableAnnotationType_e annoType;
annoType = (swTableAnnotationType_e)annoObject.Type;
if (annoType == swTableAnnotationType_e.swTableAnnotation_TitleBlock)
{
    Debug.Print("    The selected table annotation is defined in swTableAnnotationType_e as TitleBlock");
}
else
{
    Debug.Print("    The selected table annotation is defined in swTableAnnotationType_e with value: " +
annoType);
}
}
public SldWorks swApp;
}
}

```

Get Title Block Tables Example (VBA)

This example shows how to get title block tables.

```
'-----
' Preconditions:
' 1. Open a part that does not contain any annotations.
' 2. Verify that the specified table template exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inspect the output in the Immediate window.
' 2. Observe the new title block table feature under Tables
'    in the FeatureManager design tree (if necessary, right-click the
'    part in the FeatureManager design tree, click Hidden Tree
'    Items > Hide/Show Tree Items > Show in the Tables drop-down list).
' 3. Observe the corresponding title block table annotation
'    in the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim tblAnno As SldWorks.TitleBlockTableAnnotation
Dim anno As SldWorks.TitleBlockTableAnnotation
Dim tabannoObject As SldWorks.TitleBlockTableAnnotation
Dim feat As SldWorks.TitleBlockTableFeature
Dim tabfeatObject As SldWorks.TitleBlockTableFeature
Dim annos As Variant
Dim I As Long
Dim selMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Debug.Print "Inserting a title block table into the model using a general table template (*.sldtbt)"
    Debug.Print ""

    Set tblAnno = Part.Extension.InsertTitleBlockTable("C:\Program Files\SOLIDWORKS
Corp\SOLIDWORKS\lang\english\connector-table.sldtbt", 500, 500)
    Set feat = tblAnno.TitleBlockTableFeature
    Debug.Print "Title block table feature: " + feat.GetFeature.Name
    Dim count As String
    count = feat.GetTableAnnotationCount
    Debug.Print "Title block table annotation count: " + count
    Debug.Print "Title block table annotations"
    annos = feat.GetTableAnnotations
    For I = 0 To UBound(annos)
        Set anno = annos(I)
        Debug.Print "    Title block table feature: " + anno.TitleBlockTableFeature.GetFeature.Name
    Next
    Debug.Print ""

    Debug.Print "Selecting title block table feature through IModelDocExtension::SelectByID2 type,
TITLEBLOCKTABLEFEAT"
    boolstatus = Part.Extension.SelectByID2(feat.GetFeature.Name, "TITLEBLOCKTABLEFEAT", 0, 0, 0, False, 0, Nothing,
0)
    Debug.Print "    Casting selected object to ITitleBlockTableFeature"
    Set selMgr = Part.SelectionManager
    Set tabfeatObject = selMgr.GetSelectedObject6(1, -1)
    Debug.Print "    Title block table feature: " + tabfeatObject.GetFeature.Name
    Debug.Print ""

    Debug.Print "Selecting title block table annotation through IModelDocExtension::SelectByID2 type,
ANNOTATIONTABLES"
    boolstatus = Part.Extension.SelectByID2("DetailItem1@Annotations", "ANNOTATIONTABLES", -0.1205280774849,
-0.01199819470702, 0.04087038255709, False, 0, Nothing, 0)
    Debug.Print "    Casting selected object to ITitleBlockTableAnnotation type"
    Set tabannoObject = selMgr.GetSelectedObject6(1, -1)
    Debug.Print "    Getting title block table feature from the title block table annotation"
```

```

Debug.Print "      Title block table feature: " + tabannoObject.TitleBlockTableFeature.GetFeature.Name
Debug.Print ""

Debug.Print " Casting selected object to ITableAnnotation type"
Dim annoObject As ITableAnnotation
Set annoObject = selMgr.GetSelectedObject6(1, -1)
Dim annoType As String
annoType = annoObject.Type
If annoType = swTableAnnotationType_e.swTableAnnotation_TitleBlock Then
    Debug.Print "      The selected table annotation is defined in swTableAnnotationType_e as TitleBlock"
Else
    Debug.Print "      The selected table annotation is defined in swTableAnnotationType_e with value:  " +
annoType
End If
End Sub

```

Get Title Block Tables Example (VB.NET)

This example shows how to get title block tables.

```

'-----
' Preconditions:
' 1. Open a part that does not contain any annotations.
' 2. Verify that the specified table template exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inspect the output in the Immediate window.
' 2. Observe the new title block table feature under Tables
'    in the FeatureManager design tree (if necessary, right-click the
'    part in the FeatureManager design tree, click Hidden Tree
'    Items > Hide/Show Tree Items > Show in the Tables drop-down list).
' 3. Observe the corresponding title block table annotation
'    in the graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim tbtAnno As TitleBlockTableAnnotation
    Dim anno As TitleBlockTableAnnotation
    Dim tabannoObject As TitleBlockTableAnnotation
    Dim feat As TitleBlockTableFeature
    Dim tabfeatObject As TitleBlockTableFeature
    Dim annos As Object
    Dim I As Integer
    Dim selMgr As SelectionMgr
    Dim boolstatus As Boolean

    Sub main()

        Part = swApp.ActiveDoc
        Debug.Print("Inserting a title block table into the model using a general table template (*.sldtbt)")
        Debug.Print("")
        tbtAnno = Part.Extension.InsertTitleBlockTable("C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\conne
ctor-table.sldtbt", 500, 500)
        feat = tbtAnno.TitleBlockTableFeature
        Debug.Print("Title block table feature: " + feat.GetFeature.Name)
        Dim count As String
        count = feat.GetTableAnnotationCount
        Debug.Print("Title block table annotation count: " + count)
        Debug.Print("Title block table annotations")
        annos = feat.GetTableAnnotations
        For I = 0 To UBound(annos)
            anno = annos(I)
            Debug.Print("    Title block table feature: " + anno.TitleBlockTableFeature.GetFeature.Name)
        Next
        Debug.Print("")

        Debug.Print("Selecting title block table feature through IModelDocExtension::SelectByID2 type, TITLEBLOCKTABLE
FEAT")
        boolstatus = Part.Extension.SelectByID2(feat.GetFeature.Name, "TITLEBLOCKTABLEFEAT", 0, 0, 0, False, 0, Nothin
g, 0)
        Debug.Print("    Casting selected object to ITitleBlockTableFeature")
        selMgr = Part.SelectionManager
        tabfeatObject = selMgr.GetSelectedObject6(1, -1)
        Debug.Print("    Title block table feature: " + tabfeatObject.GetFeature.Name)
        Debug.Print("")

        Debug.Print("Selecting title block table annotation through IModelDocExtension::SelectByID2 type, ANNOTATIONTA
BLES")
        boolstatus = Part.Extension.SelectByID2("DetailItem1@Annotations", "ANNOTATIONTABLES", -0.1205280774849, -0.01
199819470702, 0.04087038255709, False, 0, Nothing, 0)
        Debug.Print("")
        Debug.Print("    Casting selected object to ITitleBlockTableAnnotation type")
        tabannoObject = selMgr.GetSelectedObject6(1, -1)
        Debug.Print("    Getting title block table feature from the title block table annotation")
        Debug.Print("    Title block table feature: " + tabannoObject.TitleBlockTableFeature.GetFeature.Name)
        Debug.Print("")

        Debug.Print("    Casting selected object to ITableAnnotation type")
        Dim annoObject As ITableAnnotation
        annoObject = selMgr.GetSelectedObject6(1, -1)
        Dim annoType As String
    
```

```
annoType = annoObject.Type
If annoType = swTableAnnotationType_e.swTableAnnotation_TitleBlock Then
    Debug.Print("    The selected table annotation is defined in swTableAnnotationType_e as TitleBlock")
Else
    Debug.Print("    The selected table annotation is defined in swTableAnnotationType_e with value:  " + an
noType)
End If

End Sub

Public swApp As SldWorks

End Class
```

Get_Tolerance_Example_VB (Topic)

SOLIDWORKS API Help

Get Tolerance Example (VBA)

This example shows how to get the tolerance of a model.

```
'-----  
Option Explicit  
Public Enum swTolerances_e  
    swBSCurveOutputTol = 0  
    swBSCurveNonRationalOutputTol = 1  
    swUVCurveOutputTol = 2  
    swSurfChordTessellationTol = 3  
    swSurfAngularTessellationTol = 4  
    swCurveChordTessellationTol = 5  
End Enum  
Sub main()  
    Dim swApp          As SldWorks.SldWorks  
    Dim swModeler       As SldWorks.Modeler  
  
    Set swApp = Application.SldWorks  
    Set swModeler = swApp.GetModeler  
  
    Debug.Print "Modeler Tolerances:"  
    Debug.Print "  BSCurveOutputTol          = " & swModeler.GetToleranceValue(swBSCurveOutputTol) * 1000# & " mm"  
    Debug.Print "  BSCurveNonRationalOutputTol = " & swModeler.GetToleranceValue(swBSCurveNonRationalOutputTol) * 1000# & " mm"  
    Debug.Print "  UVCurveOutputTol          = " & swModeler.GetToleranceValue(swUVCurveOutputTol) * 1000# & " mm"  
    Debug.Print "  SurfChordTessellationTol    = " & swModeler.GetToleranceValue(swSurfChordTessellationTol) * 1000# & " mm"  
    Debug.Print "  SurfAngularTessellationTol = " & swModeler.GetToleranceValue(swSurfAngularTessellationTol) * 1000# & " mm"  
    Debug.Print "  CurveChordTessellationTol  = " & swModeler.GetToleranceValue(swCurveChordTessellationTol) * 1000# & " mm"  
End Sub  
'-----
```

Get Tooling Split Feature Data Example (C#)

This example shows how to get tooling split feature data.

```
//-----
// Preconditions:
// 1. Open a model document that contains a Tooling Split1 feature.
// 2. Select Tooling Split1 in the FeatureManager design tree.
// 3. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace GetToolingSplitData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        Feature swFeat;
        ToolingSplitFeatureData swToolingSplitFeatData;
        Body2 bod;
        bool bRet;
        int i;
        object[] cavSurf;
        object[] coreSurf;
        object[] partSurf;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swToolingSplitFeatData = (ToolingSplitFeatureData)swFeat.GetDefinition();

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swFeat.Name);
            Debug.Print(" Draft angle: " + swToolingSplitFeatData.Angle);
            Debug.Print(" Depth of block in direction 1: " + swToolingSplitFeatData.get_Depth(0));
            Debug.Print(" Depth of block in direction 2: " + swToolingSplitFeatData.get_Depth(1));
            Debug.Print(" Interlock surface? " + swToolingSplitFeatData.InterlockSurface);

            bRet = swToolingSplitFeatData.AccessSelections(swModel, null);

            Debug.Print(" Cavity surfaces:");
            Debug.Print(" Count: " + swToolingSplitFeatData.GetCavitySurfacesCount());
            cavSurf = (object[])swToolingSplitFeatData.CavitySurfaces;
            for (i = 0; i <= swToolingSplitFeatData.GetCavitySurfacesCount() - 1; i++)
            {
                bod = (Body2)cavSurf[i];
                Debug.Print(" " + bod.Name);
            }
            Debug.Print(" Core surfaces:");
            Debug.Print(" Count: " + swToolingSplitFeatData.GetCoreSurfacesCount());
            coreSurf = (object[])swToolingSplitFeatData.CoreSurfaces;
            for (i = 0; i <= swToolingSplitFeatData.GetCoreSurfacesCount() - 1; i++)
```

```

    {
        bod = (Body2)coreSurf[i];
        Debug.Print("        " + bod.Name);
    }
    Debug.Print("    Parting surfaces:");
    Debug.Print("    Count: " + swToolingSplitFeatData.GetPartingSurfacesCount());
    partSurf = (object[])swToolingSplitFeatData.PartingSurfaces;
    for (i = 0; i <= swToolingSplitFeatData.GetPartingSurfacesCount() - 1; i++)
    {
        bod = (Body2)partSurf[i];
        Debug.Print("        " + bod.Name);
    }

    swToolingSplitFeatData.ReleaseSelectionAccess();

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;

}

```

Get Tooling Split Feature Data Example (VBA)

This example shows how to get tooling split feature data.

```
' -----
' Preconditions:
' 1. Open a model document that contains a Tooling Split1 feature.
' 2. Select Tooling Split1 in the FeatureManager design tree.
' 3. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
' -----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swToolingSplitFeatData As SldWorks.ToolingSplitFeatureData
Dim bod As SldWorks.Body2
Dim bRet As Boolean
Dim i As Long
Dim cavSurf As Variant
Dim coreSurf As Variant
Dim partSurf As Variant

Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swToolingSplitFeatData = swFeat.GetDefinition

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name
    Debug.Print "    Draft angle: " & swToolingSplitFeatData.Angle
    Debug.Print "    Depth of block in direction 1: " & swToolingSplitFeatData.Depth(0)
    Debug.Print "    Depth of block in direction 2: " & swToolingSplitFeatData.Depth(1)
    Debug.Print "    Interlock surface? " & swToolingSplitFeatData.InterlockSurface

    bRet = swToolingSplitFeatData.AccessSelections(swModel, Nothing)

    Debug.Print "    Cavity surfaces:"
    Debug.Print "    Count: " & swToolingSplitFeatData.GetCavitySurfacesCount
    cavSurf = swToolingSplitFeatData.CavitySurfaces
    For i = 0 To swToolingSplitFeatData.GetCavitySurfacesCount - 1
        Set bod = cavSurf(i)
        Debug.Print "        " & bod.Name
    Next
    Debug.Print "    Core surfaces:"
    Debug.Print "    Count: " & swToolingSplitFeatData.GetCoreSurfacesCount
    coreSurf = swToolingSplitFeatData.CoreSurfaces
    For i = 0 To swToolingSplitFeatData.GetCoreSurfacesCount - 1
        Set bod = coreSurf(i)
        Debug.Print "        " & bod.Name
    Next
    Debug.Print "    Parting surfaces:"
    Debug.Print "    Count: " & swToolingSplitFeatData.GetPartingSurfacesCount
    partSurf = swToolingSplitFeatData.PartingSurfaces
    For i = 0 To swToolingSplitFeatData.GetPartingSurfacesCount - 1
        Set bod = partSurf(i)
        Debug.Print "        " & bod.Name
    Next

    swToolingSplitFeatData.ReleaseSelectionAccess
```

End Sub

Get Tooling Split Feature Data Example (VB.NET)

This example shows how to get tooling split feature data.

```
'-----
' Preconditions:
' 1. Open a model document that contains a Tooling Split1 feature.
' 2. Select Tooling Split1 in the FeatureManager design tree.
' 3. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
Dim swModel As ModelDoc2
Dim swSelMgr As SelectionMgr
Dim swFeat As Feature
Dim swToolingSplitFeatData As ToolingSplitFeatureData
Dim bod As Body2
Dim bRet As Boolean
Dim i As Integer
Dim cavSurf As Object
Dim coreSurf As Object
Dim partSurf As Object

Sub main()

    swModel = swApp.ActiveDoc
    swSelMgr = swModel.SelectionManager
    swFeat = swSelMgr.GetSelectedObject6(1, -1)
    swToolingSplitFeatData = swFeat.GetDefinition

    Debug.Print("File = " & swModel.GetPathName)
    Debug.Print(" " & swFeat.Name)
    Debug.Print("    Draft angle: " & swToolingSplitFeatData.Angle)
    Debug.Print("    Depth of block in direction 1: " & swToolingSplitFeatData.Depth(0))
    Debug.Print("    Depth of block in direction 2: " & swToolingSplitFeatData.Depth(1))
    Debug.Print("    Interlock surface? " & swToolingSplitFeatData.InterlockSurface)

    bRet = swToolingSplitFeatData.AccessSelections(swModel, Nothing)

    Debug.Print("    Cavity surfaces:")
    Debug.Print("    Count: " & swToolingSplitFeatData.GetCavitySurfacesCount)
    cavSurf = swToolingSplitFeatData.CavitySurfaces
    For i = 0 To swToolingSplitFeatData.GetCavitySurfacesCount - 1
        bod = cavSurf(i)
        Debug.Print("        " & bod.Name)
    Next
    Debug.Print("    Core surfaces:")
    Debug.Print("    Count: " & swToolingSplitFeatData.GetCoreSurfacesCount)
    coreSurf = swToolingSplitFeatData.CoreSurfaces
    For i = 0 To swToolingSplitFeatData.GetCoreSurfacesCount - 1
        bod = coreSurf(i)
        Debug.Print("        " & bod.Name)
    Next
    Debug.Print("    Parting surfaces:")
    Debug.Print("    Count: " & swToolingSplitFeatData.GetPartingSurfacesCount)
    partSurf = swToolingSplitFeatData.PartingSurfaces
```

```
For i = 0 To swToolingSplitFeatData.GetPartingSurfacesCount - 1
    bod = partSurf(i)
    Debug.Print("      " & bod.Name)
Next
```

```
swToolingSplitFeatData.ReleaseSelectionAccess()
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Get Top-level Components Using Component IDs Example (C#)

This example shows how to get the top-level components in an assembly using their component IDs.

```
//-----
// Preconditions:
// 1. Open an assembly document containing subassemblies.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the assembly.
// 2. Gets the name and component ID of each top-level component
//    in the assembly.
// 3. Adds each component ID to an array and traverses the array.
//    a. Gets each top-level component using its component ID.
//    b. Gets the name and component ID of each top-level component
//       in the assembly.
// 4. Examine the Immediate window and FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            ConfigurationManager swConfMgr = default(ConfigurationManager);
            Configuration swConf = default(Configuration);
            Component2 swRootComp = default(Component2);
            object[] vChildComp = null;
            Component2 swChildComp = default(Component2);
            int i = 0;
            int compID = 0;
            int[] compIDs = null;
            int nbrComp = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssemblyDoc = (AssemblyDoc)swModel;
            swConfMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConf = (Configuration)swConfMgr.ActiveConfiguration;
            swRootComp = (Component2)swConf.GetRootComponent3(true);

            Debug.Print("Get top-level components by traversing assembly:");
            vChildComp = (object[])swRootComp.GetChildren();
            nbrComp = vChildComp.Length;
            compIDs = new int[nbrComp + 1];
            for (i = 0; i < nbrComp; i++)
            {
                swChildComp = (Component2)vChildComp[i];
                compID = swChildComp.GetID();
                Debug.Print("  Component name: " + swChildComp.Name2 + ", Component ID: " + compID);
                // Add component ID to an array
                compIDs[i] = compID;
            }

            Debug.Print("");

            Debug.Print("Get top-level components using component IDs:");
            swChildComp = null;
            for (i = 0; i < compIDs.Length - 1; i++)
            {
                compID = compIDs[i];
                swChildComp = (Component2)swAssemblyDoc.GetComponentByID(compID);
                Debug.Print("  Component name: " + swChildComp.Name2 + ", Component ID: " + swChildComp.GetID());
            }
        }
    }
}

/// <summary>
```

```
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

Get Top-level Components Using Component IDs Example (VBA)

This example shows how to get the top-level components in an assembly using their component IDs.

```
'-----
' Preconditions:
' 1. Open an assembly document containing subassemblies.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the assembly.
' 2. Gets the name and component ID of each top-level component
'    in the assembly.
' 3. Adds each component ID to an array and traverses the array.
'    a. Gets each top-level component using its component ID.
'    b. Gets the name and component ID of each top-level component
'       in the assembly.
' 4. Examine the Immediate window and FeatureManager design tree.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssemblyDoc As SldWorks.AssemblyDoc
    Dim swConfMgr As SldWorks.ConfigurationManager
    Dim swConf As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim i As Long
    Dim compID As Long
    Dim compIDs() As Long
    Dim nbrComp As Long

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swAssemblyDoc = swModel
    Set swConfMgr = swModel.ConfigurationManager
    Set swConf = swConfMgr.ActiveConfiguration
    Set swRootComp = swConf.GetRootComponent3(True)

    Debug.Print "Get top-level components by traversing assembly:"
    vChildComp = swRootComp.GetChildren
    nbrComp = UBound(vChildComp)
    ReDim compIDs(nbrComp)
    For i = 0 To nbrComp
        Set swChildComp = vChildComp(i)
        compID = swChildComp.GetID
        Debug.Print "  Component name: " & swChildComp.Name2 & ", Component ID: " & compID
        ' Add component ID to an array
        compIDs(i) = compID
    Next i

    Debug.Print ""

    Debug.Print "Get top-level components using component IDs:"
    Set swChildComp = Nothing
    nbrComp = UBound(compIDs)
    For i = 0 To UBound(compIDs)
        compID = compIDs(i)
        Set swChildComp = swAssemblyDoc.GetComponentByID(compID)
        Debug.Print "  Component name: " & swChildComp.Name2 & ", Component ID: " & swChildComp.GetID
    Next i

End Sub
```

Get Top-level Components Using Component IDs Example (VB.NET)

This example shows how to get the top-level components in an assembly using their component IDs.

```
'-----
' Preconditions:
' 1. Open an assembly document containing subassemblies.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the assembly.
' 2. Gets the name and component ID of each top-level component
'    in the assembly.
' 3. Adds each component ID to an array and traverses the array.
'    a. Gets each top-level component using its component ID.
'    b. Gets the name and component ID of each top-level component
'       in the assembly.
' 4. Examine the Immediate window and FeatureManager design tree.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssemblyDoc As AssemblyDoc
        Dim swConfMgr As ConfigurationManager
        Dim swConf As Configuration
        Dim swRootComp As Component2
        Dim vChildComp As Object
        Dim swChildComp As Component2
        Dim i As Integer
        Dim compID As Integer
        Dim compIDs() As Integer
        Dim nbrComp As Integer

        swModel = swApp.ActiveDoc
        swAssemblyDoc = swModel
        swConfMgr = swModel.ConfigurationManager
        swConf = swConfMgr.ActiveConfiguration
        swRootComp = swConf.GetRootComponent3(True)

        Debug.Print("Get top-level components by traversing assembly:")
        vChildComp = swRootComp.GetChildren
        nbrComp = UBound(vChildComp)
        ReDim compIDs(nbrComp)
        For i = 0 To nbrComp
            swChildComp = vChildComp(i)
            compID = swChildComp.GetID
            Debug.Print("  Component name: " & swChildComp.Name2 & ", Component ID: " & compID)
            ' Add component ID to an array
            compIDs(i) = compID
        Next i

        Debug.Print("")

        Debug.Print("Get top-level components using component IDs:")
        swChildComp = Nothing
        nbrComp = UBound(compIDs)
        For i = 0 To UBound(compIDs)
            compID = compIDs(i)
            swChildComp = swAssemblyDoc.GetComponentByID(compID)
            Debug.Print("  Component name: " & swChildComp.Name2 & ", Component ID: " & swChildComp.GetID)
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get the Total Number of Columns and Rows in a Table Example (C#)

This example shows how to get the total number of columns and rows (visible and hidden) in a table annotation.

```
//-----
// Preconditions: Open a drawing that contains a hole table
// with multiple columns and rows.
//
// Postconditions: Inspect the Immediate Window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace AnnotationCounts_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void ProcessTable(SldWorks swApp, ModelDoc2 swModel, TableAnnotation swTable)
        {
            Annotation swAnn = default(Annotation);
            int nNumCol = 0;
            int nNumRow = 0;
            int nTotalNumCol = 0;
            int nTotalNumRow = 0;
            swAnn = (Annotation)swTable.GetAnnotation();

            // Show the name and type of table
            Debug.Print(" " + swAnn.GetName() + " <" + swTable.Type + ">");

            // Get the visible counts
            nNumCol = swTable.ColumnCount;
            Debug.Print("    Number of visible columns: " + nNumCol);
            nNumRow = swTable.RowCount;
            Debug.Print("    Number of visible rows: " + nNumRow);

            // Get the total (hidden + visible) counts
            nTotalNumCol = swTable.TotalColumnCount;
            Debug.Print("    Total number of visible + hidden columns: " + nTotalNumCol);
            nTotalNumRow = swTable.TotalRowCount;
            Debug.Print("    Total number of visible + hidden rows: " + nTotalNumRow);

            // Hide the first row and column
            Debug.Print("");
            Debug.Print("    First row and column are now hidden");
            Debug.Print("");
            swTable.set_RowHidden(0, true);
            swTable.set_ColumnHidden(0, true);

            // Get the visible counts
            nNumCol = swTable.ColumnCount;
            Debug.Print("    Number of visible columns: " + nNumCol);
            nNumRow = swTable.RowCount;
            Debug.Print("    Number of visible rows: " + nNumRow);

            // Get the total (hidden + visible) counts
            nTotalNumCol = swTable.TotalColumnCount;
            Debug.Print("    Total number of visible + hidden columns: " + nTotalNumCol);
            nTotalNumRow = swTable.TotalRowCount;
            Debug.Print("    Total number of visible + hidden rows: " + nTotalNumRow);
        }
    }
}
```

```

public void Main()
{
    ModelDoc2 swModel = default(ModelDoc2);
    DrawingDoc swDraw = default(DrawingDoc);
    View swView = default(View);
    TableAnnotation swTable = default(TableAnnotation);

    swModel = (ModelDoc2)swApp.ActiveDoc;
    swDraw = (DrawingDoc)swModel;

    Debug.Print("File = " + swModel.GetPathName());

    // Get the first view
    swView = (View)swDraw.GetFirstView();

    while ((swView != null))
    {
        // Show the name of the view
        Debug.Print("View = " + swView.Name);

        // Get the first table annotation for this view
        swTable = swView.GetFirstTableAnnotation();

        while ((swTable != null))
        {
            ProcessTable(swApp, swModel, swTable);

            // Get next table annotation for this view
            swTable = swTable.GetNext();

        }

        // Get the next view
        swView = (View)swView.GetNextView();
    }
}

public SldWorks swApp;
}

}

```

Get the Total Number of Columns and Rows in a Table Example (VBA)

This example shows how to get the total number of columns and rows (visible and hidden) in a table annotation.

```

'-----
' Preconditions: Open a drawing that contains a hole table
' with multiple columns and rows.
'
' Postconditions: Inspect the Immediate Window.
'-----

Option Explicit

Sub ProcessTable( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swTable As SldWorks.TableAnnotation _
)

    Dim swAnn As SldWorks.Annotation
    Dim nNumCol As Long
    Dim nNumRow As Long
    Dim nTotalNumCol As Long
    Dim nTotalNumRow As Long
    Set swAnn = swTable.GetAnnotation

    ' Show the name and type of table
    Debug.Print "    " & swAnn.GetName & " <" & swTable.Type & ">"

    ' Get the visible counts
    nNumCol = swTable.ColumnCount
    Debug.Print "        Number of visible columns: " & nNumCol
    nNumRow = swTable.RowCount
    Debug.Print "        Number of visible rows: " & nNumRow

    ' Get the total (hidden + visible) counts
    nTotalNumCol = swTable.TotalColumnCount
    Debug.Print "        Total number of visible + hidden columns: " & nTotalNumCol
    nTotalNumRow = swTable.TotalRowCount
    Debug.Print "        Total number of visible + hidden rows: " & nTotalNumRow

    ' Hide the first row and column
    Debug.Print ""
    Debug.Print "        First row and column are now hidden"
    Debug.Print ""
    swTable.RowHidden(0) = True
    swTable.ColumnHidden(0) = True

    ' Get the visible counts
    nNumCol = swTable.ColumnCount
    Debug.Print "        Number of visible columns: " & nNumCol
    nNumRow = swTable.RowCount
    Debug.Print "        Number of visible rows: " & nNumRow

    ' Get the total (hidden + visible) counts
    nTotalNumCol = swTable.TotalColumnCount
    Debug.Print "        Total number of visible + hidden columns: " & nTotalNumCol
    nTotalNumRow = swTable.TotalRowCount
    Debug.Print "        Total number of visible + hidden rows: " & nTotalNumRow

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim swTable As SldWorks.TableAnnotation

```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swDraw = swModel

Debug.Print "File = " & swModel.GetPathName

' Get the first view
Set swView = swDraw.GetFirstView
Do While Not swView Is Nothing
    ' Show the name of the view
    Debug.Print "View = " & swView.Name

    ' Get the first table annotation for this view
    Set swTable = swView.GetFirstTableAnnotation

    Do While Not swTable Is Nothing
        ProcessTable swApp, swModel, swTable

        ' Get next table annotation for this view
        Set swTable = swTable.GetNext
    Loop

    ' Get the next view
    Set swView = swView.GetNextView
Loop
End Sub
```

Get the Total Number of Columns and Rows in a Table Example (VB.NET)

This example shows how to get the total number of columns and rows (visible and hidden) in a table annotation.

```
'-----  
' Preconditions: Open a drawing that contains a hole table  
' with multiple columns and rows.  
'-----  
' Postconditions: Inspect the Immediate Window.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Sub ProcessTable(ByVal swApp As SldWorks, _  
        ByVal swModel As ModelDoc2, _  
        ByVal swTable As TableAnnotation _  
    )  
  
        Dim swAnn As Annotation  
        Dim nNumCol As Long  
        Dim nNumRow As Long  
        Dim nTotalNumCol As Long  
        Dim nTotalNumRow As Long  
        swAnn = swTable.GetAnnotation  
  
        ' Show the name and type of table  
        Debug.Print("    " & swAnn.GetName & " <" & swTable.Type & ">")  
  
        ' Get the visible counts  
        nNumCol = swTable.ColumnCount  
        Debug.Print("        Number of visible columns: " & nNumCol)  
        nNumRow = swTable.RowCount  
        Debug.Print("        Number of visible rows: " & nNumRow)  
  
        ' Get the total (hidden + visible) counts  
        nTotalNumCol = swTable.TotalColumnCount  
        Debug.Print("        Total number of visible + hidden columns: " & nTotalNumCol)  
        nTotalNumRow = swTable.TotalRowCount  
        Debug.Print("        Total number of visible + hidden rows: " & nTotalNumRow)  
  
        ' Hide the first row and column  
        Debug.Print("")  
        Debug.Print("        First row and column are now hidden")  
        Debug.Print("")  
        swTable.RowHidden(0) = True  
        swTable.ColumnHidden(0) = True  
  
        ' Get the visible counts  
        nNumCol = swTable.ColumnCount  
        Debug.Print("        Number of visible columns: " & nNumCol)  
        nNumRow = swTable.RowCount  
        Debug.Print("        Number of visible rows: " & nNumRow)  
  
        ' Get the total (hidden + visible) counts  
        nTotalNumCol = swTable.TotalColumnCount  
        Debug.Print("        Total number of visible + hidden columns: " & nTotalNumCol)  
        nTotalNumRow = swTable.TotalRowCount  
        Debug.Print("        Total number of visible + hidden rows: " & nTotalNumRow)  
  
    End Sub  
  
    Sub main()  

```

```

Dim swModel As ModelDoc2
Dim swDraw As DrawingDoc
Dim swView As View
Dim swTable As TableAnnotation

swModel = swApp.ActiveDoc
swDraw = swModel

Debug.Print("File = " & swModel.GetPathName)

' Get the first view
swView = swDraw.GetFirstView
Do While Not swView Is Nothing

    ' Show the name of the view
    Debug.Print("View = " & swView.Name)

    ' Get the first table annotation for this view
    swTable = swView.GetFirstTableAnnotation

    Do While Not swTable Is Nothing
        ProcessTable(swApp, swModel, swTable)

        ' Get next table annotation for this view
        swTable = swTable.GetNext

    Loop

    ' Get the next view
    swView = swView.GetNextView

Loop

```

End Sub

```
Public swApp As SldWorks
```

End Class

Get Transform for Each Circular Pattern Instance Example (C#)

This example shows how to get the transform for each instance in a circular pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects the circular-pattern feature.
// 3. Get the number of instances in the circular-pattern
//    feature.
// 4. Gets the transform for each instance
//    in the circular-pattern feature.
// 5. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            CircularPatternFeatureData swCircularPatternFeatureData = default(CircularPatternFeatureData);
            MathTransform swMathTransform = default(MathTransform);
            bool boolstatus = false;
            int nErrors = 0;
            int nWarnings = 0;
            int NbrInstances = 0;
            int i = 0;

            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\introtosw\\pressure_plate.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.
swOpenDocOptions_Silent, "", ref nErrors, ref nWarnings);
            swModelDocExtension = (ModelDocExtension)swModel.Extension;

            // Select the circular-pattern feature
            boolstatus = swModelDocExtension.SelectByID("CirPattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swCircularPatternFeatureData = (CircularPatternFeatureData)swFeature.GetDefinition();

            // Get the number of instances in the circular-pattern feature
            NbrInstances = swCircularPatternFeatureData.TotalInstances;
            Debug.Print("Number of instances: " + NbrInstances);

            // Get the transform for each instance
            // in the circular-pattern feature
            for (i = 0; i < NbrInstances; i++)
            {
                Debug.Print("Processing instance " + (i + 1) + "...");
                swMathTransform = (MathTransform)swCircularPatternFeatureData.GetTransform(i);
                // TODO: Do something with the transform
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Get Transform for Each Circular Pattern Instance Example (VBA)

This example shows how to get the transform for each instance in a circular pattern feature.

```
'-----
' Preconditions:
' 1. Verify that the specified part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects the circular-pattern feature.
' 3. Get the number of instances in the circular-pattern
'    feature.
' 4. Gets the transform for each instance
'    in the circular-pattern feature.
' 5. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExtension As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swCircularPatternFeatureData As SldWorks.CircularPatternFeatureData
Dim swMathTransform As SldWorks.MathTransform
Dim boolstatus As Boolean
Dim nErrors As Long
Dim nWarnings As Long
Dim NbrInstances As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\introsw\pressure_plate.sldprt", swDocPART, swOpenDocOptions_Silent, "", nErrors, nWarnings)
    Set swModelDocExtension = swModel.Extension

    ' Select the circular-pattern feature
    boolstatus = swModelDocExtension.SelectByID("CirPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swCircularPatternFeatureData = swFeature.GetDefinition

    ' Get the number of instances in the circular-pattern feature
    NbrInstances = swCircularPatternFeatureData.TotalInstances
    Debug.Print "Number of instances: " & NbrInstances

    ' Get the transform for each instance
    ' in the circular-pattern feature
    For i = 0 To (NbrInstances - 1)
        Debug.Print " Processing instance " & (i + 1) & "..."
        Set swMathTransform = swCircularPatternFeatureData.GetTransform(i)
        ' TODO: Do something with the transform
    Next

End Sub
```

Get Transform for Each Circular Pattern Instance Example (VB.NET)

This example shows how to get the transform for each instance in a circular pattern feature.

```
'-----
' Preconditions:
' 1. Verify that the specified part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects the circular-pattern feature.
' 3. Get the number of instances in the circular-pattern
'   feature.
' 4. Gets the transform for each instance
'   in the circular-pattern feature.
' 5. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExtension As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swCircularPatternFeatureData As CircularPatternFeatureData
        Dim swMathTransform As MathTransform
        Dim boolstatus As Boolean
        Dim nErrors As Integer
        Dim nWarnings As Integer
        Dim NbrInstances As Integer
        Dim i As Integer

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\introtosw\pressure_plate.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptio
ns_Silent, "", nErrors, nWarnings)
        swModelDocExtension = swModel.Extension

        ' Select the circular-pattern feature
        boolstatus = swModelDocExtension.SelectByID2("CirPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swCircularPatternFeatureData = swFeature.GetDefinition

        ' Get the number of instances in the circular-pattern feature
        NbrInstances = swCircularPatternFeatureData.TotalInstances
        Debug.Print("Number of instances: " & NbrInstances)

        ' Get the transform for each instance
        ' in the circular-pattern feature
        For i = 0 To (NbrInstances - 1)
            Debug.Print(" Processing instance " & (i + 1) & "...")
            swMathTransform = swCircularPatternFeatureData.GetTransform(i)
            ' TODO: Do something with the transform
        Next
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Transform of Coordinate System Example (VB.NET)

This example shows how to get the transform of a coordinate system.

NOTE: Because VB.NET does not directly support VARIANTS, you must cast the returned objects to the appropriate data types.

```
'-----  
Private Sub API_Test2()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.feature  
    Dim vXformArr() As Double  
    Dim vXform As Object  
    Dim sAxisName As String  
    Dim bRet As Boolean  
  
    swApp = CType(CreateObject("SldWorks.Application"), SldWorks.SldWorks)  
    swModel = swApp.IActiveDoc2  
    swSelMgr = swModel.ISelectionManager  
    swFeat = CType(swSelMgr.GetSelectedObject6(1, 0), SldWorks.feature)  
    System.Diagnostics.Debug.WriteLine("File = " & swModel.GetPathName)  
    System.Diagnostics.Debug.WriteLine(" Current coordinate system = " &  
swModel.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileSaveAsCoordinateSystem))  
    System.Diagnostics.Debug.WriteLine("")  
    sAxisName = swFeat.Name  
    vXformArr = CType(swModel.GetCoordinateSystemXformByName(sAxisName), Double())  
    System.Diagnostics.Debug.WriteLine(" " & sAxisName)  
    For Each vXform In vXformArr  
        Next  
        System.Diagnostics.Debug.WriteLine(" Origin = (" + _  
            Str(-1.0# * vXformArr(9) * 1000.0#) + ", " + _  
            Str(-1.0# * vXformArr(10) * 1000.0#) + ", " + _  
            Str(-1.0# * vXformArr(11) * 1000.0#) + _  
            ") mm")  
    End Sub  
'-----
```

Get Transform of Plane Example (VBA)

This example shows how to get the transform of a plane.

```
'-----  
' Preconditions:  
' 1. Open a part that has a reference plane.  
' 2. Select the reference plane.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the transform of the selected reference plane.  
' 2. Examine the Immediate window.  
'  
' NOTE: Transform is from model origin to reference plane origin.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
    Dim swRefPlane As SldWorks.RefPlane  
    Dim swXform As SldWorks.MathTransform  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject5(1)  
    Set swRefPlane = swFeat.GetSpecificFeature2  
    Set swXform = swRefPlane.Transform  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " Plane = " & swFeat.Name  
    Debug.Print "   Origin = (" & -1# * swXform.ArrayData(9) * 1000# & ", " & -1# * swXform.ArrayData(10) * 1000# &  
    Debug.Print "   Rot1  = (" & swXform.ArrayData(0) & ", " & swXform.ArrayData(1) & ", " & swXform.ArrayData(2) &  
    Debug.Print "   Rot2  = (" & swXform.ArrayData(3) & ", " & swXform.ArrayData(4) & ", " & swXform.ArrayData(5) &  
    Debug.Print "   Rot3  = (" & swXform.ArrayData(6) & ", " & swXform.ArrayData(7) & ", " & swXform.ArrayData(8) &  
    Debug.Print "   Trans = (" & swXform.ArrayData(9) * 1000# & ", " & swXform.ArrayData(10) * 1000# & ", " &  
    Debug.Print "   Scale = " & swXform.ArrayData(12)  
  
End Sub
```

Get Transforms of Assembly Components Example (VBA)

This example shows how to recursively retrieve the transform for each child component in an assembly.

```
'-----  
'  
' Preconditions:  
' (1) Assembly is open.  
' (2) Assembly is fully resolved.  
'  
' Postconditions: None  
'  
' Notes:  
' (1) Root component does not have a name  
'  
' (2) Root component has a NULL transform, that is, no rotation,  
' translation, or scaling  
'  
' (3) All child component transforms are relative to their  
' root component  
'  
'-----  
Option Explicit  
Sub OutputCompXform  
    swComp As SldWorks.Component2  
    nLevel As Long  
  
    Dim vChild          As Variant  
    Dim swChildComp     As SldWorks.Component2  
    Dim sPadStr         As String  
    Dim swCompXform     As SldWorks.MathTransform  
    Dim vXform          As Variant  
    Dim i               As Long  
  
    For i = 0 To nLevel  
        sPadStr = sPadStr & " "  
    Next i  
  
    ' Null for root component  
    Set swCompXform = swComp.Transform2  
    If Not swCompXform Is Nothing Then  
        vXform = swCompXform.ArrayData  
  
        ' Root component has no name  
        Debug.Print sPadStr & "Component = " & swComp.Name2 & " (" & swComp.ReferencedConfiguration & ")"  
        Debug.Print sPadStr & " Suppr  = " & swComp.IsSuppressed  
        Debug.Print sPadStr & " Hidden  = " & swComp.IsHidden(False)  
  
        Debug.Print sPadStr & " Rot1  = (" & Str(vXform(0)) & ", " & Str(vXform(1)) & ", " & Str(vXform(2)) & ")"  
  
        Debug.Print sPadStr & " Rot2  = (" & Str(vXform(3)) & ", " & Str(vXform(4)) & ", " & Str(vXform(5)) & ")"
```

```

Debug.Print sPadStr & " Rot3 = (" + Str(vXform(6)) + ", " + Str(vXform(7)) + ", " + Str(vXform(8)) + ")"

Debug.Print sPadStr & " Trans = (" + Str(vXform(9)) + ", " + Str(vXform(10)) + ", " + Str(vXform(11)) + ")"

Debug.Print sPadStr & " Scale = " + Str(vXform(12))

Debug.Print ""
End If

' Recurse into subassembly
vChild = swComp.GetChildren
For i = 0 To UBound(vChild)
    Set swChildComp = vChild(i)

    OutputCompXform swChildComp, nLevel + 1
Next i
End Sub

Sub main()
    Dim swApp          As SldWorks.SldWorks
    Dim swModel         As SldWorks.ModelDoc2
    Dim swConf          As SldWorks.configuration
    Dim swRootComp      As SldWorks.Component2
    Dim bRet            As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swConf = swModel.GetActiveConfiguration
    Set swRootComp = swConf.GetRootComponent

    Debug.Print "File = " & swModel.GetPathName

    OutputCompXform swRootComp, 0
End Sub

'-----

```

Get Feature Type and Name Example (C#)

This example shows how to get the feature type and name of the selected feature for use with `IModelDocExtension::SelectByID2`.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\flopress\ball_valve\ball_valve.sldasm.  
// 2. Expand any component in the FeatureManager design tree  
//    and select one of its features.  
//  
// Postconditions:  
// 1. Gets the selected feature's type and name.  
// 2. Examine the Immediate window.  
//  
// NOTE: Because this assembly document is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace GetNameForSelectionFeatureCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            SelectionMgr swSelMgr = default(SelectionMgr);  
            Feature swFeat = default(Feature);  
            string featName = null;  
            string featType = null;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
  
            // Get the selected feature  
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);  
  
            swModel.ClearSelection2(true);  
  
            // Get the feature's type and name  
            featName = swFeat.GetNameForSelection(out featType);  
            swModelDocExt.SelectByID2(featName, featType, 0, 0, 0, true, 0, null, 0);  
  
            // Print the feature's type and name  
            // to the Immediate window  
            Debug.Print("Feature type: " + featType);  
            Debug.Print("Feature name: " + featName);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
  
        public SldWorks swApp;  
    }  
}
```

Get Type of Instant3D Feature Example (C#)

This example shows how to get the types and persistent IDs of underlying features of Instant3D features.

```
//-----  
// Preconditions:  
// 1. Verify that the specified part exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Opens the part and traverses the FeatureManager  
//    design tree.  
// 2. Gets the types of features, including the types  
//    of underlying features and persistent IDs  
//    of Instant3D features.  
// 3. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
  
namespace Instant3DFeatureCSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            Feature swFeature = default(Feature);  
            string fileName = null;  
            int errors = 0;  
            int warnings = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\block.sldprt";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART,  
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
  
            swFeature = (Feature)swModel.FirstFeature();  
  
            SelectFeat(swFeature);  
  
        }  
  
        public bool SelectFeat(Feature featureTemp)  
        {  
            while ((featureTemp != null))  
            {  
                string featureName = null;  
                featureName = featureTemp.GetTypeName2();  
  
                Debug.Print(featureName);  
  
                // Instant3D features are ICE features  
                if (featureName == "ICE")  
                {  
                    Debug.Print("    Type:    " + featureTemp.GetTypeName());  
                    Debug.Print("    ID:      " + featureTemp.GetID());  
                }  
  
                featureTemp = (Feature)featureTemp.GetNextFeature();  
            }  
            return true;  
        }  
    }  
}
```

```
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Get Type of Instant3D Feature Example (VBA)

This example shows how to get the types and persistent IDs of underlying features of Instant3D features.

```
'-----
' Preconditions:
' 1. Verify that the specified part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part and traverses the FeatureManager
'    design tree.
' 2. Gets the types of features, including the types
'    of underlying features and persistent IDs
'    of Instant3D features.
' 3. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeature As SldWorks.Feature
Dim fileName As String
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

    Set swFeature = swModel.FirstFeature

    Call SelectFeat(swFeature)

End Sub

Public Function SelectFeat(featureTemp As SldWorks.Feature) As Boolean

    While Not featureTemp Is Nothing

        Dim featureName As String
        featureName = featureTemp.GetTypeName2

        Debug.Print featureName

        ' Instant3D features are ICE features
        If featureName = "ICE" Then
            Debug.Print "    Type:    " & featureTemp.GetTypeName
            Debug.Print "    ID:      " & featureTemp.GetID
        End If

        Set featureTemp = featureTemp.GetNextFeature

    Wend

End Function
```

Get Type of Instant3D Feature Example (VB.NET)

This example shows how to get the types and persistent IDs of underlying features of Instant3D features.

```
' -----  
' Preconditions:  
' 1. Verify that the specified part exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the part and traverses the FeatureManager  
   ' design tree.  
' 2. Gets the types of features, including the types  
   ' of underlying features and the persistent  
   ' IDs of Instant3D features.  
' 3. Examine the Immediate window.  
' -----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Public Sub Main()  
  
        Dim swModel As ModelDoc2  
        Dim swFeature As Feature  
        Dim fileName As String  
        Dim errors As Integer  
        Dim warnings As Integer  
  
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\block.sldprt"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
  
        swFeature = swModel.FirstFeature  
  
        Call SelectFeat(swFeature)  
  
    End Sub  
  
    Public Function SelectFeat(ByVal featureTemp As Feature) As Boolean  
  
        While Not featureTemp Is Nothing  
  
            Dim featureName As String  
            featureName = featureTemp.GetTypeName2  
  
            Debug.Print(featureName)  
  
            ' Instant3D features are ICE features  
            If featureName = "ICE" Then  
                Debug.Print("    Type:    " & featureTemp.GetTypeName)  
                Debug.Print("    ID:      " & featureTemp.GetID)  
            End If  
  
            featureTemp = featureTemp.GetNextFeature  
  
        End While  
  
    End Function  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>  
Public swApp As SldWorks
```

End Class

Get Type of Split Line Example (C#)

This example shows how to get the type of split line.

```
//-----  
// Preconditions:  
// 1. Open a part document with a split line feature.  
// 2. Select the split line feature.  
// 3. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets the type of split line.  
// 2. Examine the Immediate window.  
//-----  
  
using Microsoft.VisualBasic;  
  
using System;  
  
using System.Collections;  
  
using System.Collections.Generic;  
  
using System.Data;  
  
using System.Diagnostics;  
  
using SolidWorks.Interop.sldworks;  
  
using SolidWorks.Interop.swconst;  
  
using System.Runtime.InteropServices;  
  
namespace GetSplitLineType_CSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swModel;  
  
        SelectionMgr swSelMgr;  
  
        Feature swFeat;  
  
        SplitLineFeatureData swSplitLineFeat;  
  
        int splittype;  
  
        public void Main()  
        {  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
  
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1,  
-1);  
  
            swSplitLineFeat = (SplitLineFeatureData)swFeat.GetDefinition();  
  
            splittype = swSplitLineFeat.GetType();  
  
            Debug.Print("Split line type (swSplitLineFeatureType_e): " + splittype);  
        }  
    }  
}
```

```
}
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Get Type of Split Line Example (VBA)

This example shows how to get the type of split line.

```
'-----  
' Preconditions:  
' 1. Open a part document with a split line feature.  
' 2. Select the split line feature.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the type of split line.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swSplitLineFeat As SldWorks.SplitLineFeatureData  
Dim splittype As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSplitLineFeat = swFeat.GetDefinition  
  
    splittype = swSplitLineFeat.GetType  
    Debug.Print "Split line type (swSplitLineFeatureType_e): " & splittype  
End Sub
```

Get Type of Split Line Example (VB.NET)

This example shows how to get the type of split line.

```
'-----  
' Preconditions:  
' 1. Open a part document with a split line feature.  
' 2. Select the split line feature.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the type of split line.  
' 2. Examine the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2  
    Dim swSelMgr As SelectionMgr  
    Dim swFeat As Feature  
    Dim swSplitLineFeat As SplitLineFeatureData  
    Dim splittype As Integer
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swFeat = swSelMgr.GetSelectedObject6(1, -1)  
        swSplitLineFeat = swFeat.GetDefinition
```

```
        splittype = swSplitLineFeat.GetType  
        Debug.Print("Split line type (swSplitLineFeatureType_e): " & splittype)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Types of Entities Attached to Selected Annotation Example (C#)

This example shows how to get the types of entities attached to a selected annotation.

```
//-----
// Preconditions:
// 1. Open a part with annotations.
// 2. Select an annotation for which you want to get attached entities.
// 3. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace GetAttachedEntities_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Note swSelObj = null;
            Annotation swAnn = default(Annotation);
            int[] vAttEntTypeArr = null;
            object[] vAttEntArr = null;
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelObj = (Note)swSelMgr.GetSelectedObject6(1, -1);
            swAnn = (Annotation)swSelObj.GetAnnotation();

            Debug.Print("Name = " + swAnn.GetName());
            Debug.Print(" Selection Type as defined in swSelectType_e = " +
            swSelMgr.GetSelectedObjectType3(1, -1));
            Debug.Print(" Annotation Type as defined in swAnnotationType_e = " + swAnn.GetType());

            vAttEntArr = (object[])swAnn.GetAttachedEntities3();
            vAttEntTypeArr = (int[])swAnn.GetAttachedEntityTypes2();

            if ((vAttEntTypeArr != null))
            {
                for (i = 0; i <= vAttEntTypeArr.GetUpperBound(0); i++)
                {
                    // A dangling annotation has at least one entity of type swSelNOTHING
                    Debug.Print(" Type of attached entity(" + i + ") as defined in swSelectType_e =
                    " + vAttEntTypeArr[i]);
                }
            }

            public SldWorks swApp;
        }
    }
}
```

Get Types of Entities Attached to Selected Annotation Example (VBA)

This example shows how to get the types of entities attached to a selected annotation.

```
'-----  
' Preconditions:  
' 1. Open a part with annotations.  
' 2. Select an annotation for which you want to get attached entities.  
' 3. Open an Immediate window.  
'  
' Postconditions: Inspect the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelObj As Object  
    Dim swAnn As SldWorks.Annotation  
    Dim vAttEntTypeArr As Variant  
    Dim vAttEntArr As Variant  
    Dim i As Long  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swSelObj = swSelMgr.GetSelectedObject6(1, -1)  
    Set swAnn = swSelObj.GetAnnotation  
  
    Debug.Print "Name = " & swAnn.GetName  
    Debug.Print " Selection Type as defined in swSelectType_e = " &  
swSelMgr.GetSelectedObjectType3(1, -1)  
    Debug.Print " Annotation Type as defined in swAnnotationType_e = " & swAnn.GetType  
  
    vAttEntArr = swAnn.GetAttachedEntities3  
    vAttEntTypeArr = swAnn.GetAttachedEntityTypes2  
  
    If Not IsEmpty(vAttEntTypeArr) Then  
        For i = 0 To UBound(vAttEntTypeArr)  
            ' A dangling annotation has at least one entity of type swSelNOTHING  
            Debug.Print " Type of attached entity(" & i & ") as defined in swSelectType_e = " &  
vAttEntTypeArr(i)  
            Next i  
        End If  
    End Sub
```

Get Types of Entities Attached to Selected Annotation Example (VB.NET)

This example shows how to get the types of entities attached to a selected annotation.

```
'-----  
' Preconditions:  
' 1. Open a part with annotations.  
' 2. Select an annotation for which you want to get attached entities.  
' 3. Open an Immediate window.  
'  
' Postconditions: Inspect the Immediate window.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swSelObj As Object  
        Dim swAnn As Annotation  
        Dim vAttEntTypeArr As Object  
        Dim vAttEntArr As Object  
        Dim i As Integer  
  
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swSelObj = swSelMgr.GetSelectedObject6(1, -1)  
        swAnn = swSelObj.GetAnnotation  
  
        Debug.Print("Name = " & swAnn.GetName)  
        Debug.Print(" Selection Type as defined in swSelectType_e = " &  
swSelMgr.GetSelectedObjectType3(1, -1))  
        Debug.Print(" Annotation Type as defined in swAnnotationType_e = " & swAnn.GetType)  
  
        vAttEntArr = swAnn.GetAttachedEntities3  
        vAttEntTypeArr = swAnn.GetAttachedEntityTypes2  
  
        If Not IsNothing(vAttEntTypeArr) Then  
            For i = 0 To UBound(vAttEntTypeArr)  
                ' A dangling annotation has at least one entity of type swSelNOTHING  
                Debug.Print(" Type of attached entity(" & i & ") as defined in swSelectType_e = " &  
vAttEntTypeArr(i))  
            Next i  
        End If  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Get Unique Name of Section View Example (C#)

This example shows how to get the unique name of a section view.

```
//-----
// Preconditions:
// 1. Open a drawing document with three views,
//    two of which are section views with the
//    same display name.
// 2. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window
// for the display names and internal unique names
// of the section views.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace UniqueNameCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            Sheet swSheet = default(Sheet);
            View swView = default(View);
            bool boolstat = false;
            SelectionMgr selMgr = default(SelectionMgr);
            View sectionView = default(View);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            selMgr = (SelectionMgr)swModel.SelectionManager;
            swDraw = (DrawingDoc)swModel;
            swSheet = (Sheet)swDraw.GetCurrentSheet();

            // Get the section views
            swView = (View)swDraw.GetFirstView();
            while ((swView != null))
            {
                if ((swView.Type == (int)swDrawingViewTypes_e.swDrawingSectionView))
                {
                    // Print the section view's display name
                    Debug.Print("Display name: " + swView.GetName2());

                    swModel.ClearSelection2(true);

                    boolstat = swModel.Extension.SelectByID2(swView.GetUniqueName(), "DRAWINGVIEW", 0, 0, 0, false, 0,
null, 0);

                    sectionView = (View)selMgr.GetSelectedObject6(1, -1);

                    // Print the section view's unique name
                    Debug.Print("Unique name: " + sectionView.GetUniqueName());

                    swModel.ClearSelection2(true);
                    Debug.Print("");
                }

                swView = (View)swView.GetNextView();
            }
        }
    }
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>
```

```
public SldWorks swApp;
```

```
    }  
}
```

Get Unique Name of Section View Example (VBA)

This example shows how to get the unique name of a section view.

```
'-----  
' Preconditions:  
' 1. Open a drawing document with three views,  
'    two of which are section views with the  
'    same display name.  
' 2. Open the Immediate window.  
'  
' Postconditions: Inspect the Immediate window  
' for the display names and internal unique names  
' of the section views.  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swDraw               As SldWorks.DrawingDoc  
    Dim swSheet              As SldWorks.Sheet  
    Dim swView               As SldWorks.view  
    Dim boolstat             As Boolean  
    Dim selMgr               As SldWorks.SelectionMgr  
    Dim view                 As SldWorks.view  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set selMgr = swModel.SelectionManager  
    Set swDraw = swModel  
    Set swSheet = swDraw.GetCurrentSheet  
  
    ' Get the section views  
    Set swView = swDraw.GetFirstView  
    While (Not swView Is Nothing)  
        If (swView.Type = swDrawingSectionView) Then  
  
            ' Print the section view's display name  
            Debug.Print "Display name: " & swView.GetName2  
  
            swModel.ClearSelection2 True  
  
            boolstat = swModel.Extension.SelectByID2(swView.GetUniqueName, "DRAWINGVIEW", 0, 0, 0,  
False, 0, Nothing, 0)  
            Set view = selMgr.GetSelectedObject6(1, -1)  
  
            ' Print the section view's unique name  
            Debug.Print "Unique name: " & view.GetUniqueName  
            swModel.ClearSelection2 True  
            Debug.Print ""  
        End If  
  
        Set swView = swView.GetNextView  
    Wend  
  
End Sub
```

Get_Unique_Name_of_Section_View_Example_VBNET (Topic)

SOLIDWORKS API Help

Get Unique Name of Section View Example (VB.NET)

This example shows how to get the unique name of a section view.

```
'-----
' Preconditions:
' 1. Open a drawing document with three views,
'    two of which are section views with the
'    same display name.
' 2. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window
' for the display names and internal unique names
' of the section views.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swSheet As Sheet
        Dim swView As View
        Dim boolstat As Boolean
        Dim selMgr As SelectionMgr
        Dim sectionView As View

        swModel = swApp.ActiveDoc
        selMgr = swModel.SelectionManager
        swDraw = swModel
        swSheet = swDraw.GetCurrentSheet

        ' Get the section views
        swView = swDraw.GetFirstView
        While (Not swView Is Nothing)
            If (swView.Type = swDrawingViewTypes_e.swDrawingSectionView) Then

                ' Print the section view's display name
                Debug.Print("Display name: " & swView.GetName2)

                swModel.ClearSelection2(True)

                boolstat = swModel.Extension.SelectByID2(swView.GetUniqueName, "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

                sectionView = selMgr.GetSelectedObject6(1, -1)

                ' Print the section view's unique name
                Debug.Print("Unique name: " & sectionView.GetUniqueName)

                swModel.ClearSelection2(True)
                Debug.Print("")
            End If

            swView = swView.GetNextView
        End While

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Get_Unopened_Document_Dependents_Example_VB (Topic)

SOLIDWORKS API Help

Get Unopened Document Dependents Example (VBA)

This example shows how to return all of the dependent files for the specified closed document. You can run this program on any type of SOLIDWORKS Design document.

```
'-----  
Option Explicit  
  
Sub main()  
    Const sDefaultName As String = "D:\docs\claw-mechanism.sldasm"  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim sDocName As String  
    Dim vDepend As Variant  
    Dim bRet As Boolean  
    Dim i As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    If Not swModel Is Nothing Then  
        sDocName = swModel.GetPathName  
    Else  
        sDocName = sDefaultName  
    End If  
  
    vDepend = swApp.GetDocumentDependencies2(sDocName, True, True, False)  
  
    Debug.Print sDocName  
    If IsEmpty(vDepend) Then  
        Debug.Print "    No dependencies"  
        Exit Sub  
    End If  
  
    For i = 0 To (UBound(vDepend) - 1) / 2  
        Debug.Print "    " + vDepend(2 * i) + " --> " + vDepend(2 * i + 1)  
    Next i  
End Sub  
'-----
```

Get_Upper-right_Text_Coordinates_of_Selected_Note_Example_VB (Topic)

SOLIDWORKS API Help

Get Upper-right Text Coordinates of Selected Note Example (VBA)

This example shows how to get the coordinates for the upper-right text point in a selected note.

```
'-----  
' Preconditions:  
' 1. Open a model document containing a note.  
' 2. Select the note.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets:  
'   a. selected note.  
'   b. name of the note.  
'   c. note's upper-right text point's coordinates.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swNote As SldWorks.Note  
    Dim vpoints As Variant  
    Dim i As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swNote = swSelMgr.GetSelectedObject6(1, 0)  
    Debug.Print swNote.GetName  
    vpoints = swNote.GetUpperRight  
    For i = 0 To UBound(vpoints)  
        Debug.Print vpoints(i)  
    Next i  
End Sub
```

Get UV Parameters for XYZ Location Example (C#)

This example shows how to use IFace2 and ICurve interfaces to get the UV parameters from the XYZ location.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\cosmosxpress\law_hook.sldprt.  
// 2. Verify that Tools > System Options > Options > FeatureManager > Solid Bodies  
//    drop-down selection is Show.  
// 3. Expand the Solid Bodies folder and select Split Line1.  
// 4. Open the Immediate window or modify the macro to write the output to a file.  
//  
// Postconditions: Observe the output in the Immediate window.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
  
namespace ReverseEvalTestMIK_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 swModel;  
        SelectionMgr swSelMgr;  
        Face2 swFace;  
  
        public void Main()  
        {  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            double[] pntData = new double[3];  
            Body2 swBody = default(Body2);  
            Body2 procBody = default(Body2);  
            swBody = (Body2)swSelMgr.GetSelectedObject6(1, -1);  
            procBody = swBody.GetProcessedBody2(0.5, 0.5);  
            //Bypass surface split  
            swFace = (Face2)procBody.GetFirstFace();  
            //Set swFace = swBody.GetFirstFace  
            while ((swFace != null))  
            {  
                double[] uvBnds = null;  
                uvBnds = (double[])swFace.GetUVBounds();  
                double UminFace = 0;  
                double UmaxFace = 0;  
                double VminFace = 0;  
                double VmaxFace = 0;  
                UminFace = uvBnds[0];  
                UmaxFace = uvBnds[1];  
                VminFace = uvBnds[2];  
                VmaxFace = uvBnds[3];  
                Surface swSurf = default(Surface);  
                swSurf = (Surface)swFace.GetSurface();  
                uvBnds = (double[])swSurf.Parameterization();  
                double UminSurf = 0;  
                double UmaxSurf = 0;
```

```

double VminSurf = 0;
double VmaxSurf = 0;
UminSurf = uvBnds[0];
UmaxSurf = uvBnds[1];
VminSurf = uvBnds[2];
VmaxSurf = uvBnds[3];
object[] vEdges = null;
vEdges = (object[])swFace.GetEdges();
int i = 0;
for (i = 0; i <= vEdges.GetUpperBound(0); i++)
{
    Edge swEdge = default(Edge);
    swEdge = (Edge)vEdges[i];
    Curve swCurve = default(Curve);
    swCurve = (Curve)swEdge.GetCurve();
    double[] vCurveParams = null;
    vCurveParams = (double[])swEdge.GetCurveParams2();
    double startParam = 0;
    double endParam = 0;
    double incParam = 0;
    double curParam = 0;
    startParam = vCurveParams[6];
    endParam = vCurveParams[7];
    incParam = (endParam - startParam) / 10.0;
    curParam = startParam;
    while (curParam < endParam)
    {
        double[] vEdgePnt = null;
        vEdgePnt = (double[])swEdge.Evaluate(curParam);
        double[] vUVSurfParams = null;
        double UEdge = 0;
        double VEdge = 0;

        // Get the UV parameters for the point using IFace2::ReverseEvaluate
        vUVSurfParams = (double[])swFace.ReverseEvaluate(vEdgePnt[0], vEdgePnt[1], vEdgePnt[2]);
        if ((vUVSurfParams != null))
        {
            UEdge = vUVSurfParams[0];
            VEdge = vUVSurfParams[1];
            Debug.Print("Edge point: " + " x: " + vEdgePnt[0] + " y: " + vEdgePnt[1] + " z: " + vEdgePnt[2]);
            Debug.Print("U parameter returned from IFace2::ReverseEvaluate is " + UEdge);
            if (UEdge > UmaxFace | UEdge < UminFace)
            {
                Debug.Print("U param error face");
            }
            //Stop
            Debug.Print("V parameter returned from IFace2::ReverseEvaluate is " + VEdge);
            if (VEdge > VmaxFace | VEdge < VminFace)
            {
                Debug.Print("V param error face");
            }
            //Stop
        }
        else
        {
            Debug.Print("Face reverse evaluate fails - empty data");
        }
    }
}

```

```

// Get the UV parameters for the point using ICurve::ReverseEvaluate
vUVSurfParams = (double[])swSurf.ReverseEvaluate(vEdgePnt[0], vEdgePnt[1], vEdgePnt[2]);
if ((vUVSurfParams != null))
{
    UEdge = vUVSurfParams[0];
    VEdge = vUVSurfParams[1];
    Debug.Print("U parameter returned from ICurve::ReverseEvaluate is " + UEdge);
    if (UEdge > UmaxFace | UEdge < UminFace)
    {
        Debug.Print("U param error surface");
    }
    //Stop
    Debug.Print("V parameter returned from ICurve::ReverseEvaluate is " + VEdge);
    if (VEdge > VmaxFace | VEdge < VminFace)
    {
        Debug.Print("V param error surface");
    }
    //Stop
}
curParam = curParam + incParam;
}
}
swFace = (Face2)swFace.GetNextFace();
}
Debug.Print("Complete");
}
public SldWorks swApp;
}
}

```

Get UV Parameters for XYZ Location Example (VBA)

This example shows how to use IFace2 and ICurve interfaces to get the UV parameters from the XYZ location.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\cosmosxpress\law_hook.sldprt.  
' 2. Verify that Tools > System Options > Options ? FeatureManager > Solid Bodies  
' drop-down selection is Show.  
' 3. Expand the Solid Bodies folder and select Split Line1.  
' 4. Open the Immediate window or modify the macro to write the output to a file.  
'  
' Postconditions: Observe the output in the Immediate window.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Dim swModel As SldWorks.ModelDoc2
```

```
Dim swSelMgr As SldWorks.SelectionMgr
```

```
Dim swFace As SldWorks.Face2
```

```
Dim vFaceSelPt As Variant
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks
```

```
    Set swModel = swApp.ActiveDoc
```

```
    Set swSelMgr = swModel.SelectionManager
```

```
    Dim pntData(2) As Double
```

```
    Dim swBody As Body2, procBody As Body2
```

```
    Set swBody = swSelMgr.GetSelectedObject6(1, -1)
```

```
    Dim skPnt As SketchPoint
```

```
    Dim swFeat As Feature
```

```
    'Set skPnt = swSelMgr.GetSelectedObject6(2, -1)
```

```
    Set procBody = swBody.GetProcessedBody2(0.5, 0.5)
```

```
    'Bypass surface split
```

```
    Set swFace = procBody.GetFirstFace
```

```
    'Set swFace = swBody.GetFirstFace
```

```
    While Not swFace Is Nothing
```

```
        Dim uvBnds As Variant
```

```
        uvBnds = swFace.GetUVBounds()
```

```
        Dim UminFace As Double, UmaxFace As Double, VminFace As Double, VmaxFace As Double
```

```
        UminFace = uvBnds(0): UmaxFace = uvBnds(1): VminFace = uvBnds(2): VmaxFace = uvBnds(3)
```

```
        Dim swSurf As Surface
```

```
        Set swSurf = swFace.GetSurface
```

```
        uvBnds = swSurf.Parameterization
```

```
        Dim UminSurf As Double, UmaxSurf As Double, VminSurf As Double, VmaxSurf As Double
```

```
        UminSurf = uvBnds(0): UmaxSurf = uvBnds(1): VminSurf = uvBnds(2): VmaxSurf = uvBnds(3)
```

```
        Dim vEdges As Variant
```

```
        vEdges = swFace.GetEdges
```

```
        Dim i As Integer
```

```

For i = 0 To UBound(vEdges)

    Dim swEdge As Edge
    Set swEdge = vEdges(i)

    Dim swCurve As SldWorks.Curve
    Set swCurve = swEdge.GetCurve()

    Dim vCurveParams As Variant
    vCurveParams = swEdge.GetCurveParams2()

    Dim startParam As Double, endParam As Double, incParam As Double, curParam As Double
    startParam = vCurveParams(6)
    endParam = vCurveParams(7)

    incParam = (endParam - startParam) / 10#
    curParam = startParam

    While curParam < endParam

        Dim vEdgePnt As Variant
        vEdgePnt = swEdge.Evaluate(curParam)

        Dim vSurfRevEval As Variant
        Dim vUVSurfParams As Variant
        Dim UEdge As Double, VEdge As Double

        ' Get the UV parameters for the point using IFace2::ReverseEvaluate
        vUVSurfParams = swFace.ReverseEvaluate(vEdgePnt(0), vEdgePnt(1), vEdgePnt(2))

        If Not IsEmpty(vUVSurfParams) Then
            UEdge = vUVSurfParams(0): VEdge = vUVSurfParams(1)

            Debug.Print "Edge point: " & vbCrLf & " x: " & vEdgePnt(0) & vbCrLf & " y: " & vEdgePnt(1) & vbCrLf & " z: " & vEdgePnt(2)
            Debug.Print "U parameter returned from IFace2::ReverseEvaluate is " & UEdge
            If UEdge > UmaxFace Or UEdge < UminFace Then
                Debug.Print "U param error face"
                'Stop
            End If

            Debug.Print "V parameter returned from IFace2::ReverseEvaluate is " & VEdge
            If VEdge > VmaxFace Or VEdge < VminFace Then
                Debug.Print "V param error face"
                'Stop
            End If
        Else
            Debug.Print "Face reverse evaluate fails - empty data"
        End If

        ' Get the UV parameters for the point using ICurve::ReverseEvaluate
        vUVSurfParams = swSurf.ReverseEvaluate(vEdgePnt(0), vEdgePnt(1), vEdgePnt(2))
        If Not IsEmpty(vUVSurfParams) Then
            UEdge = vUVSurfParams(0): VEdge = vUVSurfParams(1)

            Debug.Print "U parameter returned from ICurve::ReverseEvaluate is " & UEdge
            If UEdge > UmaxFace Or UEdge < UminFace Then

```

```
        Debug.Print "U param error surface"
        'Stop
    End If

    Debug.Print "V parameter returned from ICurve::ReverseEvaluate is " & VEdge & vbCrLf
    If VEdge > VmaxFace Or VEdge < VminFace Then
        Debug.Print "V param error surface"
        'Stop
    End If

    End If
    curParam = curParam + incParam
Wend

Next

    Set swFace = swFace.GetNextFace
Wend
Debug.Print "Complete"
End Sub
```

Get UV Parameters for XYZ Location Example (VB.NET)

This example shows how to use IFace2 and ICurve interfaces to get the UV parameters from the XYZ location.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\cosmosxpress\aw_hook.sldprt.  
' 2. Verify that Tools > System Options > Options ? FeatureManager > Solid Bodies  
'    drop-down selection is Show.  
' 3. Expand the Solid Bodies folder and select Split Line1.  
' 4. Open the Immediate window or modify the macro to write the output to a file.  
'  
' Postconditions: Observe the output in the Immediate window.  
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Imports System.Diagnostics

Partial Class SolidWorksMacro

Dim swModel As ModelDoc2

Dim swSelMgr As SelectionMgr

Dim swFace As Face2

Dim vFaceSelPt As Object

Sub main()

swModel = swApp.ActiveDoc

swSelMgr = swModel.SelectionManager

Dim pntData(2) As Double

Dim swBody As Body2, procBody As Body2

swBody = swSelMgr.GetSelectedObject6(1, -1)

procBody = swBody.GetProcessedBody2(0.5, 0.5)

'Bypass surface split

swFace = procBody.GetFirstFace

'Set swFace = swBody.GetFirstFace

While Not swFace Is Nothing

Dim uvBnds As Object

uvBnds = swFace.GetUVBounds()

Dim UminFace As Double, UmaxFace As Double, VminFace As Double, VmaxFace As Double

UminFace = uvBnds(0) : UmaxFace = uvBnds(1) : VminFace = uvBnds(2) : VmaxFace = uvBnds(3)

Dim swSurf As Surface

swSurf = swFace.GetSurface

uvBnds = swSurf.Parameterization

Dim UminSurf As Double, UmaxSurf As Double, VminSurf As Double, VmaxSurf As Double

UminSurf = uvBnds(0) : UmaxSurf = uvBnds(1) : VminSurf = uvBnds(2) : VmaxSurf = uvBnds(3)

Dim vEdges As Object

vEdges = swFace.GetEdges

Dim i As Integer

For i = 0 To UBound(vEdges)

Dim swEdge As Edge

swEdge = vEdges(i)

Dim swCurve As Curve

swCurve = swEdge.GetCurve()

Dim vCurveParams As Object

```

vCurveParams = swEdge.GetCurveParams2()
Dim startParam As Double, endParam As Double, incParam As Double, curParam As Double
startParam = vCurveParams(6)
endParam = vCurveParams(7)
incParam = (endParam - startParam) / 10.0#
curParam = startParam
While curParam < endParam
    Dim vEdgePnt As Object
    vEdgePnt = swEdge.Evaluate(curParam)
    Dim vUVSurfParams As Object
    Dim UEdge As Double, VEdge As Double
    ' Get the UV parameters for the point using IFace2::ReverseEvaluate
    vUVSurfParams = swFace.ReverseEvaluate(vEdgePnt(0), vEdgePnt(1), vEdgePnt(2))
    If Not IsNothing(vUVSurfParams) Then
        UEdge = vUVSurfParams(0) : VEdge = vUVSurfParams(1)
        Debug.Print("Edge point: " & vbCrLf & " x: " & vEdgePnt(0) & vbCrLf & " y: " & vEdgePnt(1) & vbCrLf & " z: " & vEdgePnt(2))
        Debug.Print("U parameter returned from IFace2::ReverseEvaluate is " & UEdge)
        If UEdge > UmaxFace Or UEdge < UminFace Then
            Debug.Print("U param error face")
            'Stop
        End If
        Debug.Print("V parameter returned from IFace2::ReverseEvaluate is " & VEdge)
        If VEdge > VmaxFace Or VEdge < VminFace Then
            Debug.Print("V param error face")
            'Stop
        End If
    Else
        Debug.Print("Face reverse evaluate fails - empty data")
    End If
    ' Get the UV parameters for the point using ICurve::ReverseEvaluate
    vUVSurfParams = swSurf.ReverseEvaluate(vEdgePnt(0), vEdgePnt(1), vEdgePnt(2))
    If Not IsNothing(vUVSurfParams) Then
        UEdge = vUVSurfParams(0) : VEdge = vUVSurfParams(1)
        Debug.Print("U parameter returned from ICurve::ReverseEvaluate is " & UEdge)
        If UEdge > UmaxFace Or UEdge < UminFace Then
            Debug.Print("U param error surface")
            'Stop
        End If
        Debug.Print("V parameter returned from ICurve::ReverseEvaluate is " & VEdge & vbCrLf)
        If VEdge > VmaxFace Or VEdge < VminFace Then
            Debug.Print("V param error surface")
            'Stop
        End If
    End If
    curParam = curParam + incParam
End While
Next
swFace = swFace.GetNextFace
End While
Debug.Print("Complete")
End Sub
Public swApp As SldWorks
End Class

```

Get Version History Example (VBA)

This example shows how to get the version history of a SOLIDWORKS Design model document.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the version number of the model document.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim vVerStr As Variant  
    Dim i As Long  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    If IsEmpty(vVerStr) Then  
        vVerStr = swApp.VersionHistory(swModel.GetPathName)  
    End If  
  
    If Not IsEmpty(vVerStr) Then  
        For i = 0 To UBound(vVerStr)  
            Debug.Print "  " & vVerStr(i)  
        Next i  
    Else  
        Debug.Print "  No version information."  
    End If  
  
End Sub
```

Get Version History of Future Version Document Example (C#)

This example shows how to get the version history of a future version document.

```
//-----
// Preconditions:
// 1. Copy a future version part document to this macro's directory.
// 2. Replace Part1.sldprt in the macro with the name
//    of your future version part.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. Does not throw swFileLoadError_e.swFutureVersion.
// 2. Inspect the Immediate window for the version history of the document.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace IsFutureVersion_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swDocument = default(ModelDoc2);
            ModelDocExtension swExtension = default(ModelDocExtension);
            int lErrors = 0;
            int lWarnings = 0;
            int lOptions = 0;
            string strFileName = null;
            object[] vVersionHistory = null;
            string strBaseVersion = null;
            string strCurrentVersion = null;
            string strHotFixes = null;
            string strRevisionNumber = null;
            int lDateCode = 0;
            string strFormatVersion = null;

            strFormatVersion = swApp.GetLatestSupportedFileVersion().ToString();

            strRevisionNumber = swApp.RevisionNumber();
            swApp.GetBuildNumbers2(out strBaseVersion, out strCurrentVersion, out strHotFixes);
            lDateCode = swApp.DateCode();

            Debug.Print("Format version = " + strFormatVersion);
            Debug.Print("RevisionNumber = " + strRevisionNumber);
            Debug.Print("BaseVersion = " + strBaseVersion);
            Debug.Print("CurrentVersion = " + strCurrentVersion);
            Debug.Print("HotFixes = " + strHotFixes);
            Debug.Print("DateCode = " + lDateCode);

            swDocument = (ModelDoc2)swApp.ActiveDoc;

            if ((swDocument == null))
            {
                strFileName = swApp.GetCurrentMacroPathFolder() + "\\Part1.sldprt";
                lOptions = 0;
                lOptions = lOptions | (int)swOpenDocOptions_e.swOpenDocOptions_Silent;
```

```

        swDocument = swApp.OpenDoc6(strFileName, (int)swDocumentTypes_e.swDocPART, lOptions,
"", ref lErrors, ref lWarnings);
        Debug.Print("lErrors = " + lErrors);

        // Starting with SW2012 SP5, loading future file versions
        // is supported, so the future version error no longer occurs.
        Debug.Print(" future version error = " + ((lErrors &
(int)swFileLoadError_e.swFutureVersion) == (int)swFileLoadError_e.swFutureVersion));
        Debug.Print("lWarnings = " + lWarnings);
    }

    if ((swDocument == null))
    {
        Debug.Print("No model");
        return;
    }

    strFileName = swDocument.GetPathName();
    Debug.Print("File = " + strFileName);
    swExtension = swDocument.Extension;

    // The version history of a future version document is the same
    // as that of the part/assembly template that is used to load it
    // into the older version of SOLIDWORKS Design. IModelDoc2::VersionHistory
    // returns the version history of the part template,
    // not the version history of the future version document.
    // Get the version history of a future version document from
    // its file on disk using SldWorks::VersionHistory.

    Debug.Print("Is future version = " + swExtension.IsFutureVersion());

    if (!(swExtension.IsFutureVersion() == false))
    {
        vVersionHistory = (object[])swApp.VersionHistory(strFileName);
        if ((vVersionHistory != null))
        {
            Debug.Print("Version history from file:");
            PrintVersionHistory(vVersionHistory);
        }
    }

    vVersionHistory = (object[])swDocument.VersionHistory();

    if ((vVersionHistory != null))
    {
        Debug.Print("Version history from document:");
        PrintVersionHistory(vVersionHistory);
    }

    Debug.Print("view-only      = " + swDocument.IsOpenedViewOnly());
    Debug.Print("read-only      = " + swDocument.IsOpenedReadOnly());
    Debug.Print("blocking state = " +
BlockingState2String((swBlockingStates_e)swDocument.GetBlockingState()));

}

private void PrintVersionHistory(object[] vVersionHistory)
{
    object[] vSplitResults = null;
    string strFormatVersion = null;
    string strDateCodes = null;
    string vDateCode = null;
    string vHistoryEntry = null;

    foreach (string vHistoryEntry_loopVariable in vVersionHistory)
    {
        vHistoryEntry = vHistoryEntry_loopVariable;

        Debug.Print(" " + vHistoryEntry);
        vSplitResults = vHistoryEntry.Split('|');
    }
}

```

```

        strFormatVersion = (string)vSplitResults[0];
        strDateCodes = vSplitResults[1].ToString().Replace(']', ' ');
        vSplitResults = strDateCodes.Split(',');
        Debug.Print("    format version = " + strFormatVersion);

        foreach (string vDateCode_loopVariable in vSplitResults)
        {
            vDateCode = vDateCode_loopVariable;
            Debug.Print("        datecode = " + vDateCode);
        }

    }

}

private string BlockingState2String(swBlockingStates_e nBlockingState)
{
    string functionReturnValue = null;

    switch ((nBlockingState))
    {
        case swBlockingStates_e.swEditorBlock:
            functionReturnValue = "editor block";

            break;

        case swBlockingStates_e.swEditSketchBlock:
            functionReturnValue = "edit sketch block";

            break;

        case swBlockingStates_e.swFullBlock:
            functionReturnValue = "full block";

            break;

        case swBlockingStates_e.swModifyBlock:
            functionReturnValue = "modify block";

            break;

        case swBlockingStates_e.swNoBlock:
            functionReturnValue = "no block";

            break;

        case swBlockingStates_e.swPartialModifyBlock:
            functionReturnValue = "partial modify block";

            break;

        case swBlockingStates_e.swSystemBlock:
            functionReturnValue = "system block";

            break;

        case swBlockingStates_e.swViewOnlyBlock:
            functionReturnValue = "view only block";

            break;

        default:
            functionReturnValue = "<unknown blocking state>";

            break;
    }

    return functionReturnValue;
}

```

```
        public SldWorks swApp;  
    }  
}
```

Get Version History of Future Version Document Example (VBA)

This example shows how to get the version history of a future version document.

```
' -----  
' Preconditions:  
' 1. Copy a future version part document to this macro's directory.  
' 2. Replace Part1.sldprt in the macro with the name  
'    of your future version part.  
' 3. Open an Immediate window.  
'  
' Postconditions:  
' 1. Does not throw swFileLoadError_e.swFutureVersion.  
' 2. Inspect the Immediate window for the version history of the document.  
' -----
```

Option Explicit

Sub Main()

```
Dim swApp As SldWorks.SldWorks  
Dim swDocument As SldWorks.ModelDoc2  
Dim swExtension As SldWorks.ModelDocExtension  
Dim lErrors As Long  
Dim lWarnings As Long  
Dim lOptions As Long  
Dim strFileName As String  
Dim vVersionHistory As Variant  
Dim strBaseVersion As String  
Dim strCurrentVersion As String  
Dim strHotFixes As String  
Dim strRevisionNumber As String  
Dim lDateCode As Long  
Dim strDateCode As String  
Dim strDateCodeYear As String  
Dim strDateCodeDay As String  
Dim strFormatVersion As String  
  
Set swApp = Application.SldWorks  
  
strFormatVersion = swApp.GetLatestSupportedFileVersion  
  
strRevisionNumber = swApp.RevisionNumber  
swApp.GetBuildNumbers2 strBaseVersion, strCurrentVersion, strHotFixes  
lDateCode = swApp.DateCode  
  
strDateCode = CStr(lDateCode)  
strDateCodeYear = Left$(strDateCode, 4)  
strDateCodeDay = Mid$(strDateCode, Len(strDateCodeYear) + 1)  
  
strDateCode = strDateCodeYear & "/" & strDateCodeDay  
  
Debug.Print "Format version = " & strFormatVersion  
Debug.Print "RevisionNumber = " & strRevisionNumber  
Debug.Print "BaseVersion = " & strBaseVersion  
Debug.Print "CurrentVersion = " & strCurrentVersion  
Debug.Print "HotFixes = " & strHotFixes  
Debug.Print "DateCode = " & lDateCode  
Debug.Print " DateCode = " & strDateCode  
  
Set swDocument = swApp.ActiveDoc  
  
If (swDocument Is Nothing) Then  
    strFileName = swApp.GetCurrentMacroPathFolder & "\Part1.sldprt"
```

```

lOptions = 0
lOptions = lOptions Or swOpenDocOptions_e.swOpenDocOptions_Silent
Set swDocument = swApp.OpenDoc6(strFileName, swDocPART, lOptions, "", lErrors, lWarnings)
Debug.Print "lErrors = " & lErrors

' Starting with SW2012 SP5, loading future file versions is supported,
' so the future version error no longer occurs.
Debug.Print " future version error = " & ((lErrors And swFileLoadError_e.swFutureVersion) =
swFileLoadError_e.swFutureVersion)
Debug.Print "lWarnings = " & lWarnings
End If

If (swDocument Is Nothing) Then
    Debug.Print "No model"
    Exit Sub
End If

strFileName = swDocument.GetPathName
Debug.Print "File = " & strFileName
Set swExtension = swDocument.Extension

' The version history of a future version document is the same as
' that of the part/assembly template that is used to load it
' into the older version of SOLIDWORKS Design. IModelDoc2::VersionHistory
' returns the version history of the part template,
' not the version history of the future version document. Get the version
' history of a future version document from its file on disk
' using SldWorks::VersionHistory.

Debug.Print "Is future version = " & swExtension.IsFutureVersion

If (Not (swExtension.IsFutureVersion = False)) Then
    vVersionHistory = swApp.VersionHistory(strFileName)
    If (Not IsEmpty(vVersionHistory)) Then
        Debug.Print "Version history from file:"
        PrintVersionHistory vVersionHistory
    End If
End If

vVersionHistory = swDocument.VersionHistory

If (Not IsEmpty(vVersionHistory)) Then
    Debug.Print "Version history from document:"
    PrintVersionHistory vVersionHistory
End If

Debug.Print "view-only      = " & swDocument.IsOpenedViewOnly
Debug.Print "read-only      = " & swDocument.IsOpenedReadOnly
Debug.Print "blocking state = " & BlockingState2String(swDocument.GetBlockingState)

End Sub

Private Sub PrintVersionHistory(ByVal vVersionHistory As Variant)
    Dim vSplitResults      As Variant
    Dim strFormatVersion    As String
    Dim strDateCodes        As String
    Dim vDateCode           As Variant
    Dim vHistoryEntry       As Variant
    For Each vHistoryEntry In vVersionHistory

        Debug.Print " " & vHistoryEntry
        vSplitResults = Split(vHistoryEntry, "[")
        strFormatVersion = vSplitResults(0)
        strDateCodes = Replace(vSplitResults(1), "]", "")
        vSplitResults = Split(strDateCodes, ",")
        Debug.Print "    format version = " & strFormatVersion
    
```

```

        For Each vDateCode In vSplitResults
            Debug.Print "            datecode = " & vDateCode
        Next vDateCode

    Next vHistoryEntry
End Sub

Private Function BlockingState2String(ByVal nBlockingState As SwConst.swBlockingStates_e) As String
    Select Case (nBlockingState)
        Case swBlockingStates_e.swEditorBlock
            BlockingState2String = "editor block"

        Case swBlockingStates_e.swEditSketchBlock
            BlockingState2String = "edit sketch block"

        Case swBlockingStates_e.swFullBlock
            BlockingState2String = "full block"

        Case swBlockingStates_e.swModifyBlock
            BlockingState2String = "modify block"

        Case swBlockingStates_e.swNoBlock
            BlockingState2String = "no block"

        Case swBlockingStates_e.swPartialModifyBlock
            BlockingState2String = "partial modify block"

        Case swBlockingStates_e.swSystemBlock
            BlockingState2String = "system block"

        Case swBlockingStates_e.swViewOnlyBlock
            BlockingState2String = "view only block"

        Case Else
            BlockingState2String = "<unknown blocking state>"

    End Select
End Function

```

Get Version History of Future Version Document Example (VB.NET)

This example shows how to get the version history of a future version document.

```
' -----
' Preconditions:
' 1. Copy a future version part document to this macro's directory.
' 2. Replace Part1.sldprt in the macro with the name
'    of your future version part.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Does not throw swFileLoadError_e.swFutureVersion.
' 2. Inspect the Immediate window for the version history of the document.
' -----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub Main()
```

```
        Dim swDocument As ModelDoc2
        Dim swExtension As ModelDocExtension
        Dim lErrors As Integer
        Dim lWarnings As Integer
        Dim lOptions As Integer
        Dim strFileName As String
        Dim vVersionHistory As Object
        Dim strBaseVersion As String = Nothing
        Dim strCurrentVersion As String = Nothing
        Dim strHotFixes As String = Nothing
        Dim strRevisionNumber As String
        Dim lDateCode As Integer
        Dim strDateCode As String
        Dim strDateCodeYear As String
        Dim strDateCodeDay As String
        Dim strFormatVersion As String

        strFormatVersion = swApp.GetLatestSupportedFileVersion

        strRevisionNumber = swApp.RevisionNumber
        swApp.GetBuildNumbers2(strBaseVersion, strCurrentVersion, strHotFixes)
        lDateCode = swApp.DateCode

        strDateCode = CStr(lDateCode)
        strDateCodeYear = Left$(strDateCode, 4)
        strDateCodeDay = Mid$(strDateCode, Len(strDateCodeYear) + 1)

        strDateCode = strDateCodeYear & "/" & strDateCodeDay

        Debug.Print("Format version = " & strFormatVersion)
        Debug.Print("RevisionNumber = " & strRevisionNumber)
        Debug.Print("BaseVersion = " & strBaseVersion)
        Debug.Print("CurrentVersion = " & strCurrentVersion)
        Debug.Print("HotFixes = " & strHotFixes)
        Debug.Print("DateCode = " & lDateCode)
        Debug.Print("    DateCode = " & strDateCode)

        swDocument = swApp.ActiveDoc

        If (swDocument Is Nothing) Then
            strFileName = swApp.GetCurrentMacroPathFolder & "\Part1.sldprt"
```

```

lOptions = 0
lOptions = lOptions Or swOpenDocOptions_e.swOpenDocOptions_Silent
swDocument = swApp.OpenDoc6(strFileName, swDocumentTypes_e.swDocPART, lOptions, "",
lErrors, lWarnings)
Debug.Print("lErrors = " & lErrors)

' Starting with SW2012 SP5, loading future file versions
' is supported, so the future version error no longer occurs.
Debug.Print(" future version error = " & ((lErrors And
swFileLoadError_e.swFutureVersion) = swFileLoadError_e.swFutureVersion))
Debug.Print("lWarnings = " & lWarnings)
End If

If (swDocument Is Nothing) Then
Debug.Print("No model")
Exit Sub
End If

strFileName = swDocument.GetPathName
Debug.Print("File = " & strFileName)
swExtension = swDocument.Extension

' The version history of a future version document is the same
' as that of the part/assembly template that is used to load it
' into the older version of SOLIDWORKS Design. IModelDoc2::VersionHistory
' returns the version history of the part template,
' not the version history of the future version document.
' Get the version history of a future version document from its file
' on disk using SldWorks::VersionHistory.

Debug.Print("Is future version = " & swExtension.IsFutureVersion)

If (Not (swExtension.IsFutureVersion = False)) Then
vVersionHistory = swApp.VersionHistory(strFileName)
If (Not IsNothing(vVersionHistory)) Then
Debug.Print("Version history from file:")
PrintVersionHistory(vVersionHistory)
End If
End If

vVersionHistory = swDocument.VersionHistory

If (Not IsNothing(vVersionHistory)) Then
Debug.Print("Version history from document:")
PrintVersionHistory(vVersionHistory)
End If

Debug.Print("view-only = " & swDocument.IsOpenedViewOnly)
Debug.Print("read-only = " & swDocument.IsOpenedReadOnly)
Debug.Print("blocking state = " & BlockingState2String(swDocument.GetBlockingState))

End Sub

Private Sub PrintVersionHistory(ByVal vVersionHistory As Object)

Dim vSplitResults As Object
Dim strFormatVersion As String
Dim strDateCodes As String
Dim vDateCode As Object
Dim vHistoryEntry As Object

For Each vHistoryEntry In vVersionHistory

Debug.Print(" " & vHistoryEntry)
vSplitResults = Split(vHistoryEntry, "[")
strFormatVersion = vSplitResults(0)
strDateCodes = Replace(vSplitResults(1), "]", "")
vSplitResults = Split(strDateCodes, ",")
Debug.Print(" format version = " & strFormatVersion)

```

```
For Each vDateCode In vSplitResults
    Debug.Print("    datecode = " & vDateCode)
Next vDateCode
```

```
Next vHistoryEntry
```

```
End Sub
```

```
Private Function BlockingState2String(ByVal nBlockingState As swBlockingStates_e) As String
```

```
    Select Case (nBlockingState)
```

```
        Case swBlockingStates_e.swEditorBlock
            BlockingState2String = "editor block"
```

```
        Case swBlockingStates_e.swEditSketchBlock
            BlockingState2String = "edit sketch block"
```

```
        Case swBlockingStates_e.swFullBlock
            BlockingState2String = "full block"
```

```
        Case swBlockingStates_e.swModifyBlock
            BlockingState2String = "modify block"
```

```
        Case swBlockingStates_e.swNoBlock
            BlockingState2String = "no block"
```

```
        Case swBlockingStates_e.swPartialModifyBlock
            BlockingState2String = "partial modify block"
```

```
        Case swBlockingStates_e.swSystemBlock
            BlockingState2String = "system block"
```

```
        Case swBlockingStates_e.swViewOnlyBlock
            BlockingState2String = "view only block"
```

```
        Case Else
            BlockingState2String = "<unknown blocking state>"
```

```
    End Select
```

```
End Function
```

```
Public swApp As SldWorks
```

```
End Class
```

Get Version Number Example (C#)

This example shows how to get the version number of SOLIDWORKS Design.

```
//-----  
// Preconditions: Open the Immediate  
// window.  
//  
// Postconditions: Examine the Immediate  
// window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace VersionCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        string BaseVersion;  
        string CurrentVersion;  
        string HotFixes;  
  
        public void Main()  
        {  
            Debug.Print("Version number: " + swApp.RevisionNumber());  
            swApp.GetBuildNumbers2(out BaseVersion, out CurrentVersion, out HotFixes);  
            Debug.Print("Major revision number: " + BaseVersion);  
            Debug.Print("Build number: " + CurrentVersion);  
            Debug.Print("Hot fix numbers: " + HotFixes);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Get Version Number Example (VBA)

This example shows how to get the version number of SOLIDWORKS Design.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions: Examine the Immediate  
' window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim BaseVersion As String  
Dim CurrentVersion As String  
Dim HotFixes As String  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Debug.Print "Version = " & swApp.RevisionNumber  
    swApp.GetBuildNumbers2 BaseVersion, CurrentVersion, HotFixes  
    Debug.Print "Major revision number: " & BaseVersion  
    Debug.Print "Build number: " & CurrentVersion  
    Debug.Print "Hot fix numbers: " & HotFixes  
  
End Sub
```

Get_Version_Number_Example_VBNET (Topic)

SOLIDWORKS API Help

Get Version Number Example (VB.NET)

This example shows how to the version number of SOLIDWORKS Design.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions: Examine the Immediate  
' window.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Public Sub Main()  
  
        Dim BaseVersion As String = Nothing  
        Dim CurrentVersion As String = Nothing  
        Dim HotFixes As String = Nothing  
  
        Debug.Print("Version = " & swApp.RevisionNumber)  
        swApp.GetBuildNumbers2(BaseVersion, CurrentVersion, HotFixes)  
        Debug.Print("Major revision number: " & BaseVersion)  
        Debug.Print("Build number: " & CurrentVersion)  
        Debug.Print("Hot fix numbers: " & HotFixes)  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Get Vertex Example (VBA)

This example shows how to get the x,y,z location of each vertex on the selected face:

- Query a face for edge information
- Traverse the topology of a part from the face down to the vertex
- Determine the selected object count
- Get objects selected by the user
- Determine object types

```
'-----  
Public Sub GetEndPoints()  
  
    Dim i As Long  
    Dim Msg As String  
    Dim Style As Variant  
    Dim Title As String  
    Dim swApp, Part, SelMgr As Object  
  
    ' Face object  
    Dim faceObj As Object  
  
    ' Array of edge objects  
    Dim edgeList As Variant  
    Dim edgeCount As Long  
  
    ' Edge object  
    Dim edgeObj As Object  
  
    ' Vertex objects  
    Dim startVertexObj As Object  
    Dim endVertexObj As Object  
  
    ' Edge startpoint and endpoint arrays  
    Dim startPt As Variant  
    Dim endPt As Variant  
  
    Set swApp = CreateObject("SldWorks.Application")  
    ' Use the active document  
    Set Part = swApp.ActiveDoc  
    ' Setup to handle user selections  
    Set SelMgr = Part.SelectionManager  
  
    ' If no selection made, return an error  
    If (SelMgr.GetSelectedObjectCount = 0) Then  
        swApp.SendMsgToUser ("Select a face first...")  
  
    ' Otherwise, continue  
    Else  
  
        If (SelMgr.GetSelectedObjectType(1) <> 2) Then  
            swApp.SendMsgToUser ("Must select a face")  
            Exit Sub  
        End If  
    End If  
End Sub
```

End If

' Get the selected Face object

Set faceObj = SelMgr.**GetSelectedObject5**(1)

' Get edge count from the face

edgeCount = faceObj.**GetEdgeCount**

' Get all edges from the face

edgeList = faceObj.**GetEdges**

' For each edge

For i = 0 To (edgeCount - 1)

Set edgeObj = edgeList(i)

' Get the "Start" vertex

Set startVertexObj = edgeObj.**GetStartVertex**

' Get the "End" vertex

Set endVertexObj = edgeObj.**GetEndVertex**

' Define message

Msg = "Edge points: " + Chr(10)

If (Not startVertexObj Is Nothing) Then

' Get the xyz vertex location

startPt = startVertexObj.**GetPoint**

Msg = Msg + Str\$(startPt(0))+", "+Str\$(startPt(1))+", "+Str\$(startPt(2))+Chr(10)

End If

If (Not endVertexObj Is Nothing) Then

' Get the xyz vertex location

endPt = endVertexObj.**GetPoint**

Msg = Msg + Str\$(endPt(0))+", "+Str\$(endPt(1))+", "+Str\$(endPt(2))

End If

' OK Button only

Style = vbOKOnly

' Define title

Title = "Vertex Info"

' Display message to user

Call MsgBox(Msg, Style, Title)

Next

End If

End Sub

Get View Bounding Box and Position Example (C#)

This example shows how to get the bounding box, geometric center, and position lock status of all drawing views in the drawing sheet.

```
//-----
// Preconditions:
// 1. Verify that the drawing document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing document.
// 2. Gets each drawing view's:
//    * x and y coordinates of the geometric center relative
//    * to the drawing sheet origin
//    * bounding box
//    * position lock status
// 3. Examine the Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            DrawingDoc swDraw = default(DrawingDoc);
            View swView = default(View);
            double[] outline = null;
            double[] pos = null;
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\replaceview.slddrw";
            swDraw = (DrawingDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e
.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swView = (View)swDraw.GetFirstView();
            while ((swView != null))
            {
                outline = (double[])swView.GetOutline();
                pos = (double[])swView.Position;
                Debug.Print("View = " + swView.Name);
                Debug.Print(" X and Y positions = (" + pos[0] * 1000.0 + ", " + pos[1] * 1000.0 + ") mm");
                Debug.Print(" X and Y bounding box minimums = (" + outline[0] * 1000.0 + ", " + outline[1] * 1000.0 +
") mm");
                Debug.Print(" X and Y bounding box maximums = (" + outline[2] * 1000.0 + ", " + outline[3] * 1000.0 +
") mm");
                Debug.Print(" Position locked?" + swView.PositionLocked);
                swView = (View)swView.GetNextView();
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Get View Bounding Box and Position Example (VBA)

This example shows how to get the bounding box, geometric center, and position lock status of all drawing views in the drawing sheet.

```
'-----
' Preconditions:
' 1. Verify that the drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Gets each drawing view's:
'    * x and y coordinates of the geometric center relative
'      to the drawing sheet origin
'    * bounding box
'    * position lock status
' 3. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save
' changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim outline() As Double
    Dim pos() As Double
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = CreateObject("SldWorks.Application")
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\replaceview.slddrw"
    Set swDraw = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent,
    "", errors, warnings)
    Set swView = swDraw.GetFirstView

    Do While Not swView Is Nothing
        outline = swView.GetOutline
        pos = swView.Position
        Debug.Print "View = " + swView.Name
        Debug.Print "  X and Y positions = (" & pos(0) * 1000# & ", " & pos(1) * 1000# & ") mm"
        Debug.Print "  X and Y bounding box minimums = (" & outline(0) * 1000# & ", " & outline(1) * 1000# & ") mm"
        Debug.Print "  X and Y bounding box maximums = (" & outline(2) * 1000# & ", " & outline(3) * 1000# & ") mm"
        Debug.Print "  Position locked? " & swView.PositionLocked
        Set swView = swView.GetNextView
    Loop

End Sub
```

Get View Bounding Box and Position Example (VB.NET)

This example shows how to get the bounding box, geometric center, and position lock status of all drawing views in the drawing sheet.

```
'-----
' Preconditions:
' 1. Verify that the drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Gets each drawing view's:
'    * x and y coordinates of the geometric center relative
'    * to the drawing sheet origin
'    * bounding box
'    * position lock status
' 3. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swDraw As DrawingDoc
        Dim swView As View
        Dim outline() As Double
        Dim pos() As Double
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\replaceview.slddrw"
        swDraw = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent,
"", errors, warnings)
        swView = swDraw.GetFirstView
        Do While Not swView Is Nothing
            outline = swView.GetOutline
            pos = swView.Position
            Debug.Print("View = " + swView.Name)
            Debug.Print(" X and Y positions = (" & pos(0) * 1000.0# & ", " & pos(1) * 1000.0# & ") mm")
            Debug.Print(" X and Y bounding box minimums = (" & outline(0) * 1000.0# & ", " & outline(1) * 1000.0# & "
) mm")
            Debug.Print(" X and Y bounding box maximums = (" & outline(2) * 1000.0# & ", " & outline(3) * 1000.0# & "
) mm")
            Debug.Print(" Position locked? " + swView.PositionLocked)
            swView = swView.GetNextView
        Loop

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get_View_Mode_Names_for_Model_Example_VB (Topic)

SOLIDWORKS API Help

Get View Mode Names for Model Example (VBA)

This example shows how to get all of the view mode names for the current part or assembly.

```
'-----  
'  
' Preconditions: Part or assembly document is open.  
'  
' Postconditions: None  
'  
'-----  
Option Explicit  
Sub main()  
    Dim swApp          As SldWorks.SldWorks  
    Dim swModel         As SldWorks.ModelDoc2  
    Dim vModelViewNames As Variant  
    Dim i               As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    vModelViewNames = swModel.GetModelViewNames  
  
    Debug.Print "FileName = " & swModel.GetPathName  
    Debug.Print "  ModelViewCount = " & swModel.GetModelViewCount  
    For i = 0 To UBound(vModelViewNames)  
        Debug.Print "    " + vModelViewNames(i)  
    Next i  
End Sub  
'-----
```

Get Views and Notes (VBA)

This example shows how to get all of the views and notes in a drawing document.

```
'-----  
' Preconditions: Drawing document is open and at least  
'               one view has some notes.  
'  
' Postconditions: None  
'  
' NOTE: IDrawingDoc::GetViews returns both sheets and views.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawDoc As SldWorks.DrawingDoc  
Dim swView As SldWorks.View  
Dim swNote As SldWorks.Note  
Dim sheetCount As Long  
Dim viewCount As Long  
Dim noteCount As Long  
Dim i As Long  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swDrawDoc = swModel  
  
Dim viewCount As Long  
viewCount = swDrawDoc.GetViewCount  
  
Dim ss As Variant  
ss = swDrawDoc.GetViews  
  
For sheetCount = LBound(ss) To UBound(ss)  
    Dim vv As Variant  
    vv = ss(sheetCount)  
  
    For viewCount = LBound(vv) To UBound(vv)  
        Debug.Print (vv(viewCount).GetName2())  
  
        Dim vNotes As Variant  
        noteCount = vv(viewCount).GetNoteCount
```

```
If noteCount > 0 Then
    vNotes = vv(viewCount).GetNotes
    For i = 0 To noteCount - 1
        Debug.Print " Note text: " & vNotes(i).GetText
    Next
End If
```

```
Next viewCount
```

```
Next sheetCount
```

```
End Sub
```

Get Virtual Sharp Witness Line Data Example (C#)

This example shows how to get the geometry data of all of the virtual sharp witness lines in a drawing.

```
//-----  
// Preconditions: Open the drawing of a part that contains one or more  
// virtual sharps with witness lines.  
//  
// Postconditions: Inspect the Immediate Window for virtual sharp witness line  
// geometry data.  
//-----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace GetWitnessGeomInfo_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 swModel;  
        DrawingDoc pDrawing;  
        View swView;  
        int Count;  
        int size;  
        Double[] entitiesData;  
        int index;  
        int entityCounts;  
        int entType;  
        int linesCount;  
        int arcsCount;  
        int VirtualSharpNum;  
        int j;  
  
        public void Main()  
        {  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            pDrawing = (DrawingDoc)swModel;  
  
            swView = (View)pDrawing.GetFirstView();  
  
            while ((swView != null))  
            {  
                Debug.Print("Drawing view: " + swView.Name);  
  
                Count = swView.GetWitnessEntitiesCount(out size);  
  
                Debug.Print("  Number of virtual sharp witness lines: " + Count);  
                Debug.Print("  Size of virtual sharp witness line geometry data array: " + size);  
  
                if ((Count > 0))  
                {  
                    entitiesData = (double[])swView.GetWitnessGeomInfo();  
  
                    if ((entitiesData != null))  
                    {  
                        VirtualSharpNum = 0;  
                        index = 0;  
                        entityCounts = 0;  
                        while (entityCounts < Count - 1)  
                        {  
                            Debug.Print("    Virtual sharp " + VirtualSharpNum);  
                            Debug.Print("      Color: " + entitiesData[index]);  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        index = index + 1;
        Debug.Print("        Line style (swLineStyles_e): " + entitiesData[index]);
        index = index + 1;
        Debug.Print("        Line weight: (swLineWeights_e): " +
entitiesData[index]);
        index = index + 1;
        Debug.Print("        Layer ID: " + entitiesData[index]);
        index = index + 1;
        Debug.Print("        Layer override (swLayerOverride_e):" +
entitiesData[index]);
        index = index + 1;

        entType = (int)entitiesData[index];
        Debug.Print("        Entity type: " + entType);
        if ((entType == 0))
        {
            index = index + 1;
            linesCount = (int)entitiesData[index];
            Debug.Print("        Line count: " + linesCount);
            index = index + 1;

            for (j = 0; j <= linesCount - 1; j++)
            {
                Debug.Print("        Start: x =" + entitiesData[index] + " y =" +
entitiesData[index + 1] + " z =" + entitiesData[index + 2]);
                index = index + 3;
                Debug.Print("        End:   x =" + entitiesData[index] + " y =" +
entitiesData[index + 1] + " z =" + entitiesData[index + 2]);
                index = index + 3;
            }
            entityCounts = entityCounts + linesCount;
        }

        entType = (int)entitiesData[index];
        Debug.Print("        Entity type: " + entType);
        if ((entType == 1))
        {
            index = index + 1;
            arcsCount = (int)entitiesData[index];
            Debug.Print("        Arc count: " + arcsCount);
            index = index + 1;

            for (j = 0; j <= arcsCount - 1; j++)
            {
                Debug.Print("        Start: x =" + entitiesData[index] + " y ="
+ entitiesData[index + 1] + " z =" + entitiesData[index + 2]);
                index = index + 3;
                Debug.Print("        End:   x =" + entitiesData[index] + " y ="
+ entitiesData[index + 1] + " z =" + entitiesData[index + 2]);
                index = index + 3;
                Debug.Print("        Center: x =" + entitiesData[index] + " y ="
+ entitiesData[index + 1] + " z =" + entitiesData[index + 2]);
                index = index + 3;
                Debug.Print("        Normal: x =" + entitiesData[index] + " y ="
+ entitiesData[index + 1] + " z =" + entitiesData[index + 2]);
                index = index + 3;
            }

            entityCounts = entityCounts + arcsCount;
        }

        VirtualSharpNum = VirtualSharpNum + 1;
    }
}

swView = (View)swView.GetNextView();
}
}

```

```
    public SldWorks swApp;  
}  
}
```

Get Virtual Sharp Witness Line Data Example (VBA)

This example shows how to get the geometry data of all of the virtual sharp witness lines in a drawing.

```
'-----  
' Preconditions: Open the drawing of a part that contains one or more  
' virtual sharps with witness lines.  
'  
' Postconditions: Inspect the Immediate Window for virtual sharp witness line  
' geometry data.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim pDrawing As SldWorks.DrawingDoc  
Dim swView As SldWorks.View  
Dim Count As Long  
Dim size As Long  
Dim entitiesData As Variant  
Dim index As Long  
Dim entityCounts As Long  
Dim entType As Long  
Dim linesCount As Long  
Dim arcsCount As Long  
Dim VirtualSharpNum As Long  
Dim j As Long  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set pDrawing = swModel  
  
    Set swView = pDrawing.GetFirstView  
    While Not swView Is Nothing  
  
        Debug.Print "Drawing view: "; swView.Name  
  
        Count = swView.GetWitnessEntitiesCount(size)  
  
        Debug.Print "  Number of virtual sharp witness lines: "; Count  
        Debug.Print "  Size of virtual sharp witness line geometry data array: "; size  
  
        If (Count > 0) Then  
            entitiesData = swView.GetWitnessGeomInfo  
  
            If Not IsEmpty(entitiesData) Then  
                VirtualSharpNum = 0  
                index = 0  
                entityCounts = 0  
                While entityCounts < Count - 1  
                    Debug.Print "    Virtual sharp "; VirtualSharpNum  
                    Debug.Print "    Color: "; entitiesData(index)  
                    index = index + 1  
                    Debug.Print "    Line style (swLineStyles_e): "; entitiesData(index)  
                    index = index + 1  
                    Debug.Print "    Line weight: (swLineWeights_e): "; entitiesData(index)  
                    index = index + 1  
                    Debug.Print "    Layer ID: "; entitiesData(index)  
                    index = index + 1  
                    Debug.Print "    Layer override (swLayerOverride_e):"; entitiesData(index)  
                    index = index + 1  
                    VirtualSharpNum = VirtualSharpNum + 1  
                    entityCounts = entityCounts + 1  
                End While  
            End If  
        End If  
    End While  
End Sub
```

```

entType = entitiesData(index)
Debug.Print "      Entity type: "; entType
If (entType = 0) Then
    index = index + 1
    linesCount = entitiesData(index)
    Debug.Print "      Line count: "; linesCount
    index = index + 1

    For j = 0 To linesCount - 1
        Debug.Print "      Start: x ="; entitiesData(index); " y =";
entitiesData(index + 1); " z ="; entitiesData(index + 2)
        index = index + 3
        Debug.Print "      End:   x ="; entitiesData(index); " y =";
entitiesData(index + 1); " z ="; entitiesData(index + 2)
        index = index + 3
    Next j
    entityCounts = entityCounts + linesCount
End If

entType = entitiesData(index)
Debug.Print "      Entity type: "; entType
If (entType = 1) Then
    index = index + 1
    arcsCount = entitiesData(index)
    Debug.Print "      Arc count: "; arcsCount
    index = index + 1

    For j = 0 To arcsCount - 1
        Debug.Print "      Start: x ="; entitiesData(index); " y =";
entitiesData(index + 1); " z ="; entitiesData(index + 2)
        index = index + 3
        Debug.Print "      End:   x ="; entitiesData(index); " y =";
entitiesData(index + 1); " z ="; entitiesData(index + 2)
        index = index + 3
        Debug.Print "      Center: x ="; entitiesData(index); " y =";
entitiesData(index + 1); " z ="; entitiesData(index + 2)
        index = index + 3
        Debug.Print "      Normal: x ="; entitiesData(index); " y =";
entitiesData(index + 1); " z ="; entitiesData(index + 2)
        index = index + 3
    Next j

    entityCounts = entityCounts + arcsCount

End If

VirtualSharpNum = VirtualSharpNum + 1

Wend

End If
End If

Set swView = swView.GetNextView

Wend

End Sub

```

Get Virtual Sharp Witness Line Data Example (VB.NET)

This example shows how to get the geometry data of all of the virtual sharp witness lines in a drawing.

```

'-----
' Preconditions: Open the drawing of a part that contains one or more
' virtual sharps with witness lines.
'
' Postconditions: Inspect the Immediate Window for virtual sharp witness line
' geometry data.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim pDrawing As DrawingDoc
    Dim swView As View
    Dim Count As Long
    Dim size As Long
    Dim entitiesData As Object
    Dim index As Long
    Dim entityCounts As Long
    Dim entType As Long
    Dim linesCount As Long
    Dim arcsCount As Long
    Dim VirtualSharpNum As Long
    Dim j As Long

    Sub main()

        swModel = swApp.ActiveDoc
        pDrawing = swModel

        swView = pDrawing.GetFirstView
        While Not swView Is Nothing

            Debug.Print("Drawing view: " & swView.Name)

            Count = swView.GetWitnessEntitiesCount(size)

            Debug.Print("  Number of virtual sharp witness lines: " & Count)
            Debug.Print("  Size of virtual sharp witness line geometry data array: " & size)

            If (Count > 0) Then
                entitiesData = swView.GetWitnessGeomInfo

                If Not IsNothing(entitiesData) Then
                    VirtualSharpNum = 0
                    index = 0
                    entityCounts = 0
                    While entityCounts < Count - 1
                        Debug.Print("    Virtual sharp " & VirtualSharpNum)
                        Debug.Print("      Color: " & entitiesData(index))
                        index = index + 1
                        Debug.Print("      Line style (swLineStyles_e): " & entitiesData(index))
                        index = index + 1
                        Debug.Print("      Line weight: (swLineWeights_e): " & entitiesData(index))
                        index = index + 1
                        Debug.Print("      Layer ID: " & entitiesData(index))
                        index = index + 1
                        Debug.Print("      Layer override (swLayerOverride_e):" &
entitiesData(index))
                    End While
                End If
            End If
        End While
    End Sub
End Class

```

```

        index = index + 1

        entityType = entitiesData(index)
        Debug.Print("          Entity type: " & entityType)
        If (entityType = 0) Then
            index = index + 1
            linesCount = entitiesData(index)
            Debug.Print("          Line count: " & linesCount)
            index = index + 1

            For j = 0 To linesCount - 1
                Debug.Print("          Start: x =" & entitiesData(index) & " y =" &
entitiesData(index + 1) & " z =" & entitiesData(index + 2))
                index = index + 3
                Debug.Print("          End:   x =" & entitiesData(index) & " y =" &
entitiesData(index + 1) & " z =" & entitiesData(index + 2))
                index = index + 3
            Next j
            entityCounts = entityCounts + linesCount
        End If

        entityType = entitiesData(index)
        Debug.Print("          Entity type: " & entityType)
        If (entityType = 1) Then
            index = index + 1
            arcsCount = entitiesData(index)
            Debug.Print("          Arc count: " & arcsCount)
            index = index + 1

            For j = 0 To arcsCount - 1
                Debug.Print("          Start: x =" & entitiesData(index) & " y =" &
entitiesData(index + 1) & " z =" & entitiesData(index + 2))
                index = index + 3
                Debug.Print("          End:   x =" & entitiesData(index) & " y =" &
entitiesData(index + 1) & " z =" & entitiesData(index + 2))
                index = index + 3
                Debug.Print("          Center: x =" & entitiesData(index) & " y =" &
entitiesData(index + 1) & " z =" & entitiesData(index + 2))
                index = index + 3
                Debug.Print("          Normal: x =" & entitiesData(index) & " y =" &
entitiesData(index + 1) & " z =" & entitiesData(index + 2))
                index = index + 3
            Next j

            entityCounts = entityCounts + arcsCount

        End If

        VirtualSharpNum = VirtualSharpNum + 1

    End While

    End If
End If

swView = swView.GetNextView

End While

End Sub

Public swApp As SldWorks

End Class

```

Get Visibility of and Resolve Lightweight Components Example (VBA)

This example shows how to get the visibility of and resolve lightweight components.

```
'-----
' Preconditions:
' 1. Open an assembly lightweight (click File > Open >
'   assembly_file_name.sldasm > change Mode to
'   Lightweight > Open.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the FeatureManager design tree and resolves
'   all components.
' 2. Examine the Immediate window and the FeatureManager
'   design tree.
'-----
Option Explicit

Sub GetComponentVisib(swComp As SldWorks.Component2, sPadStr As String)

    Dim vChildArray As Variant
    Dim swChildComp As SldWorks.Component2
    Dim i As Long

    ' Root component has no name (empty string)
    Debug.Print sPadStr & swComp.Name2 & " [" & swComp.Visible & "]"

    vChildArray = swComp.GetChildren
    For i = 0 To UBound(vChildArray)
        Set swChildComp = vChildArray(i)
        GetComponentVisib swChildComp, sPadStr & "  "
    Next i

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swConfig As SldWorks.Configuration
    Dim swConfMgr As SldWorks.ConfigurationManager
    Dim swRootComp As SldWorks.Component2
    Dim nStatus As Long

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swConfMgr = swModel.ConfigurationManager
    Set swConfig = swConfMgr.ActiveConfiguration
    Set swRootComp = swConfig.GetRootComponent3(True)

    nStatus = swAssy.ResolveAllLightWeightComponents(False)
    Debug.Assert swResolveOk = nStatus

    Debug.Print "File = " & swModel.GetPathName

    GetComponentVisib swRootComp, "  "

End Sub
```

Get Visible Components and Entities in Drawing View Example (C#)

This example shows how to get the visible components and entities in a drawing view.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
// 2. Select Drawing View2.
//
// Postconditions:
// 1. Gets the visible entities in Drawing View2.
// 2. Inspect the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace GetSilhouetteEdges_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            DrawingDoc swDrawing = default(DrawingDoc);
            View drView = default(View);
            Component2 Comp = default(Component2);
            SelectData selData = default(SelectData);
            Entity ent = default(Entity);
            int itr = 0;
            int CompCount = 0;
            object[] vComps = null;
            object[] vEdges = null;
            object[] vVerts = null;
            object[] vFaces = null;
            object[] vSilhouetteEdges = null;
            int i = 0;
            bool boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDrawing = (DrawingDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            drView = (View)swDrawing.ActiveDrawingView;

            Debug.Assert((drView != null));
            Debug.Print("Name of drawing view: " + drView.GetName2());
            Debug.Print("");

            CompCount = drView.GetVisibleComponentCount();

            Debug.Assert(CompCount != 0);
            Debug.Print("Number of visible components = " + CompCount);

            vComps = (object[])drView.GetVisibleComponents();

            Debug.Assert((vComps != null));

            for (i = vComps.GetLowerBound(0); i <= vComps.GetUpperBound(0); i++)
            {
                swModel.ClearSelection2(true);
                Debug.Print("");
                Debug.Print("Component " + i + " name is " + ((Component2)vComps[i]).Name2);
            }
        }
    }
}
```

```

Comp = (Component2)vComps[i];

//Get all edges of this component that are visible in this drawing view
vEdges = (object[])drView.GetVisibleEntities2(Comp,
(int)swViewEntityType_e.swViewEntityType_Edge);

selData = swSelMgr.CreateSelectData();

selData.View = drView;

if ((vEdges == null))
{
    Debug.Print("    No edges.");
}
else
{
    Debug.Print("    This component has " + vEdges.GetUpperBound(0) + 1 + " visible
edges in this view.");
    for (itr = 0; itr <= vEdges.GetUpperBound(0); itr++)
    {
        ent = (Entity)vEdges[itr];
        boolstatus = ent.Select4(false, selData);
    }
}

//Get all vertices of this component that are visible in this drawing view
vVerts = (object[])drView.GetVisibleEntities2(Comp,
(int)swViewEntityType_e.swViewEntityType_VerTEX);

if ((vVerts == null))
{
    Debug.Print("    No vertices.");
}
else
{
    Debug.Print("    This component has " + vVerts.GetUpperBound(0) + 1 + " visible
vertices in this view.");
    for (itr = 0; itr <= vVerts.GetUpperBound(0); itr++)
    {
        ent = (Entity)vVerts[itr];
        boolstatus = ent.Select4(false, selData);
    }
}

swModel.ClearSelection2(true);

//Get all faces of this component that are visible in this drawing view
vFaces = (object[])drView.GetVisibleEntities2(Comp,
(int)swViewEntityType_e.swViewEntityType_Face);

if ((vFaces == null))
{
    Debug.Print("    No faces.");
}
else
{
    Debug.Print("    This component has " + vFaces.GetUpperBound(0) + 1 + " visible
faces in this view.");
    for (itr = 0; itr <= vFaces.GetUpperBound(0); itr++)
    {
        ent = (Entity)vFaces[itr];
        boolstatus = ent.Select4(false, selData);
    }
}

//Get all silhouette edges of this component that are visible in this drawing view
vSilhouetteEdges = (object[])drView.GetVisibleEntities2(Comp,
(int)swViewEntityType_e.swViewEntityType_SilhouetteEdge);

SilhouetteEdge silEdge = default(SilhouetteEdge);
if ((vSilhouetteEdges == null))
{
    Debug.Print("    No silhouette edges.");
}

```

```

        else
        {
            Debug.Print("    This component has " + vSilhouetteEdges.GetUpperBound(0) + 1 + "
visible silhouette edges in this view.");
            for (itr = 0; itr <= vSilhouetteEdges.GetUpperBound(0); itr++)
            {
                silEdge = (SilhouetteEdge)vSilhouetteEdges[itr];
                boolstatus = silEdge.Select(false, selData);
            }
        }
    }

}

public SldWorks swApp;

}

}

```

Get Visible Components and Entities in Drawing View Example (VB.NET)

This example shows how to get the visible components and entities in a drawing view.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
' 2. Select Drawing View2.  
'  
' Postconditions:  
' 1. Gets the visible entities in Drawing View2.  
' 2. Inspect the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swDrawing As DrawingDoc  
        Dim drView As View  
        Dim Comp As Component2  
        Dim selData As SelectData  
        Dim ent As Entity  
        Dim itr As Integer  
        Dim CompCount As Integer  
        Dim vComps As Object  
        Dim vEdges As Object  
        Dim vVerts As Object  
        Dim vFaces As Object  
        Dim vSilhouetteEdges As Object  
        Dim i As Integer  
        Dim boolstatus As Boolean
```

```
        swModel = swApp.ActiveDoc  
        swDrawing = swModel  
        swSelMgr = swModel.SelectionManager  
        drView = swDrawing.ActiveDrawingView
```

```
        Debug.Assert(Not drView Is Nothing)  
        Debug.Print("Name of drawing view: " & drView.GetName2)  
        Debug.Print("")
```

```
        CompCount = drView.GetVisibleComponentCount
```

```
        Debug.Assert(CompCount <> 0)  
        Debug.Print("Number of visible components = " & CompCount)
```

```
        vComps = drView.GetVisibleComponents
```

```
        Debug.Assert(Not IsNothing(vComps))
```

```
        For i = LBound(vComps) To UBound(vComps)  
            swModel.ClearSelection2(True)  
            Debug.Print("")  
            Debug.Print("Component " & i & " name is " & vComps(i).Name2)
```

```
            Comp = vComps(i)
```

```
            'Get all edges of this component that are visible in this drawing view
```

```

vEdges = drView.GetVisibleEntities2(Comp, swViewEntityType_e.swViewEntityType_Edge)

selData = swSelMgr.CreateSelectData

selData.View = drView

If IsNothing(vEdges) Then
    Debug.Print("    No edges.")
Else
    Debug.Print("    This component has " & UBound(vEdges) + 1 & " visible edges in this
view.")
    For itr = 0 To UBound(vEdges)
        ent = vEdges(itr)
        boolstatus = ent.Select4(False, selData)
    Next itr
End If

'Get all vertices of this component that are visible in this drawing view
vVerts = drView.GetVisibleEntities2(Comp, swViewEntityType_e.swViewEntityType_Vertex)

If IsNothing(vVerts) Then
    Debug.Print("    No vertices.")
Else
    Debug.Print("    This component has " & UBound(vVerts) + 1 & " visible vertices in
this view.")
    For itr = 0 To UBound(vVerts)
        ent = vVerts(itr)
        boolstatus = ent.Select4(False, selData)
    Next itr
End If

swModel.ClearSelection2(True)

'Get all faces of this component that are visible in this drawing view
vFaces = drView.GetVisibleEntities2(Comp, swViewEntityType_e.swViewEntityType_Face)

If IsNothing(vFaces) Then
    Debug.Print("    No faces.")
Else
    Debug.Print("    This component has " & UBound(vFaces) + 1 & " visible faces in this
view.")
    For itr = 0 To UBound(vFaces)
        ent = vFaces(itr)
        boolstatus = ent.Select4(False, selData)
    Next itr
End If

'Get all silhouette edges of this component that are visible in this drawing view
vSilhouetteEdges = drView.GetVisibleEntities2(Comp,
swViewEntityType_e.swViewEntityType_SilhouetteEdge)

Dim silEdge As SilhouetteEdge
If IsNothing(vSilhouetteEdges) Then
    Debug.Print("    No silhouette edges.")
Else
    Debug.Print("    This component has " & UBound(vSilhouetteEdges) + 1 & " visible
silhouette edges in this view.")
    For itr = 0 To UBound(vSilhouetteEdges)
        silEdge = vSilhouetteEdges(itr)
        boolstatus = silEdge.Select(False, selData)
    Next itr
End If
Next i

End Sub

Public swApp As SldWorks

End Class

```

Get Visible Components and Entities in Drawing View Example (VBA)

This example shows how to get the visible components and entities in a drawing view.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
' 2. Select Drawing View2.  
'  
' Postconditions:  
' 1. Gets the visible entities in Drawing View2.  
' 2. Inspect the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swDrawing As SldWorks.DrawingDoc  
    Dim drView As SldWorks.View  
    Dim Comp As SldWorks.Component2  
    Dim selData As SldWorks.SelectData  
    Dim ent As SldWorks.Entity  
    Dim itr As Long  
    Dim CompCount As Long  
    Dim vComps As Variant  
    Dim vEdges As Variant  
    Dim vVerts As Variant  
    Dim vFaces As Variant  
    Dim vSilhouetteEdges As Variant  
    Dim i As Long  
    Dim boolstatus As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDrawing = swModel  
    Set swSelMgr = swModel.SelectionManager  
    Set drView = swDrawing.ActiveDrawingView  
  
    Debug.Assert Not drView Is Nothing  
    Debug.Print "Name of drawing view: " & drView.GetName2  
    Debug.Print  
  
    CompCount = drView.GetVisibleComponentCount  
  
    Debug.Assert CompCount <> 0  
    Debug.Print "Number of visible components = " & CompCount  
  
    vComps = drView.GetVisibleComponents  
  
    Debug.Assert Not IsEmpty(vComps)  
  
    For i = LBound(vComps) To UBound(vComps)  
        swModel.ClearSelection2 True  
        Debug.Print ""  
        Debug.Print "Component " & i & " name is " & vComps(i).Name2  
  
        Set Comp = vComps(i)  
  
        'Get all edges of this component that are visible in this drawing view  
        vEdges = drView.GetVisibleEntities2(Comp, swViewEntityType_Edge)
```

```

Set selData = swSelMgr.CreateSelectData

selData.View = drView

If IsEmpty(vEdges) Then
    Debug.Print "    No edges."
Else
    Debug.Print "    This component has " & UBound(vEdges) + 1 & " visible edges in this
view."
    For itr = 0 To UBound(vEdges)
        Set ent = vEdges(itr)
        boolstatus = ent.Select4(False, selData)
    Next itr
End If

'Get all vertices of this component that are visible in this drawing view
vVerts = drView.GetVisibleEntities2(Comp, swViewEntityType_Vertex)

If IsEmpty(vVerts) Then
    Debug.Print "    No vertices."
Else
    Debug.Print "    This component has " & UBound(vVerts) + 1 & " visible vertices in this
view."
    For itr = 0 To UBound(vVerts)
        Set ent = vVerts(itr)
        boolstatus = ent.Select4(False, selData)
    Next itr
End If

swModel.ClearSelection2 True

'Get all faces of this component that are visible in this drawing view
vFaces = drView.GetVisibleEntities2(Comp, swViewEntityType_Face)

If IsEmpty(vFaces) Then
    Debug.Print "    No faces."
Else
    Debug.Print "    This component has " & UBound(vFaces) + 1 & " visible faces in this
view."
    For itr = 0 To UBound(vFaces)
        Set ent = vFaces(itr)
        boolstatus = ent.Select4(False, selData)
    Next itr
End If

'Get all silhouette edges of this component that are visible in this drawing view
vSilhouetteEdges = drView.GetVisibleEntities2(Comp, swViewEntityType_SilhouetteEdge)

Dim silEdge As SldWorks.SilhouetteEdge
If IsEmpty(vSilhouetteEdges) Then
    Debug.Print "    No silhouette edges."
Else
    Debug.Print "    This component has " & UBound(vSilhouetteEdges) + 1 & " visible
silhouette edges in this view."
    For itr = 0 To UBound(vSilhouetteEdges)
        Set silEdge = vSilhouetteEdges(itr)
        boolstatus = silEdge.Select(False, selData)
    Next itr
End If
Next i
End Sub

```

Insert Chain Dimensions Example (C#)

This example shows how to insert chain dimensions in a drawing.

```
//=====
// Preconditions:
// 1. Open install_dir\samples\tutorial\advdrawings\foodprocessor.slddrw.
// 2. Select the Sheet2 tab at the bottom.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Observe the chained dimensions in Drawing View3.
// 2. Inspect the display dimension values in the Immediate window.
//
// NOTE: Because the sample is used elsewhere, do not save changes.
//=====
```

```
using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
```

```
namespace InsertChainDimensions_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDocExtension modDocExt;
            ModelDoc2 Part;
            SelectionMgr selectMgr;
            object[] dimArray = new object[4];
            object dimObject;
            object[] entityArray = new object[4];
            object varArray;
            DisplayDimension myDisplayDim;
```

```

    Dimension swDim;
    string dimText;
    int k;
    bool boolstatus;

    Part = (ModelDoc2)swApp.ActiveDoc;
    selectMgr = (SelectionMgr)Part.SelectionManager;
    modDocExt = Part.Extension;

    Part.ClearSelection2(true);
    boolstatus = ((DrawingDoc)Part).ActivateView("Drawing View3");
    boolstatus = Part.Extension.SelectByRay(0.107406727925462, 0.259964392021715, 375.00575,
0, 0, -1, 0.00193314752083778, 1, false, 0, 0);
    entityArray[0] = selectMgr.GetSelectedObject6(1, -1);

    boolstatus = Part.Extension.SelectByRay(0.135835367937783, 0.281001585630832, 375.00575,
0, 0, -1, 0.00193314752083778, 1, false, 0, 0);
    entityArray[1] = selectMgr.GetSelectedObject6(2, -1);

    boolstatus = Part.Extension.SelectByRay(0.140383950339754, 0.25598438241999, 375.00575,
0, 0, -1, 0.00193314752083778, 1, false, 0, 0);
    entityArray[2] = selectMgr.GetSelectedObject6(3, -1);

    boolstatus = Part.Extension.SelectByRay(0.176772609555524, 0.221301441604959, 375.00275,
0, 0, -1, 0.00193314752083778, 1, false, 0, 0);
    entityArray[3] = selectMgr.GetSelectedObject6(4, -1);

    varArray = entityArray;

    dimObject = modDocExt.InsertChainDimensions(varArray);

    dimArray = (object[])dimObject;

    if (dimArray.Length > 0)
    {
        for (k = 0; k <= dimArray.GetUpperBound(0); k++)
        {
            myDisplayDim = (DisplayDimension)dimArray[k];
            swDim = myDisplayDim.GetDimension2(0);
            dimText = swDim.Value.ToString();
            Debug.Print(dimText);
        }
    }
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}

```

